

RailCall

Compliance Program — Policy Set v1.0

Adopted 2026-07-21 by Sami Ben Chaalia (Security Officer)
Bundle generated 2026-07-21

Confidential — for review under NDA

RailCall — HIPAA Security Risk Analysis (v1)

Date written: 2026-07-21 **Applies to:** - `railcall-engine` @ `9521956fd949d956e315edb74dc434c284f4526f` (branch `feat/paid-tier-entitlement`) - `railcall-core` @ `06bdd1449eddd6b65438ca92f53273e7f3334cf6` (branch `main`) **Status:** ADOPTED 2026-07-21 by Sami Ben Chaalia (Security Officer). **Framework:** HHS SRA Tool categories aligned to 45 CFR §164.308(a)(1)(ii)(A) (Security Rule Risk Analysis). **Scope of this SRA:** the RailCall product surface — engine, station (local runtime), gateway (`railcall-core.onrender.com`), CLI, receipts and audit chain. Personal, HR, marketing, and back-office systems are OUT of scope for v1.

0. About this document

This SRA is written to be read, not signed off. Every implemented control cites the file and function that implements it. Every gap is stated as a gap with the reason and the mitigation plan. Nothing is claimed as "in progress" without a specific owner and a date.

This is version 1. It should be re-verified against the code at each substantive release (station-v0.16 was the release this document was written against) and re-adopted at least annually per §164.316(b)(2)(iii).

Two things this SRA is deliberately NOT: - **Not a compliance certification.** HIPAA has no certifying body; compliance is self-determined. Publishing this document does not make RailCall compliant. - **Not a substitute for organizational risk management.** Administrative and physical safeguards (§164.308, §164.310) are covered here at the level a small-team product can honestly claim; a healthcare deployment requires the customer's own §164.308(a)(1)(ii)(B) implementation on top.

1. Scope — the systems in play

System	What it does	Where PHI could enter
Local engine (<code>railcall-engine</code>)	User's machine, runs governed workflows on <code>127.0.0.1</code> . Signs receipts.	Any workflow field the user configures.
Station bundle (<code>railcall_station.tar.gz</code> , released via installer)	Ships the engine + workbench code to end users.	N/A (no runtime data at rest).
CLI (<code>railcall_cli.py</code> , <code>railcall_companion_daemon.py</code>)	Local commands (<code>railcall demo</code> , <code>railcall activate</code>), signs <code>railcall demo</code> receipts, holds the vault.	Not by default — the CLI does not accept clinical data.
Gateway (<code>cloud_gateway.py</code> on Render at <code>https://railcall-core.onrender.com</code>)	Mint entitlements, seat validation, Stripe webhook, meter/balance. Sees <code>sha256(api_key)</code> , <code>install_pubkey</code> , integrity HASHES from receipt bundles — never receipt bodies .	Contract violation only — see §5.3 for what forces this to be true.
Website + docs (<code>railcall-contrib/website-v2</code> , deployed to <code>railcall.ai</code>)	Marketing + docs. No PHI, no PII beyond signup email.	Signup emails only.

Key architectural fact. The engine executes locally under the customer's control. RailCall the operator never sees the customer's workflow inputs or outputs; the gateway only ever receives:

- `sha256(api_key)` for billing (`cloud_gateway.py` /v1/meter, /v1/seat/checkin — the `key_hash` path is the primary; raw keys are a legacy alternative kept for backward compatibility, both paths flow to the same DB column)
- Signed integrity hashes and attestation ids (`cloud_gateway.py` /v1/attestation/countersign)
- Stripe webhook data (payment status, subscription lifecycle)

This means for the majority of RailCall deployments, **RailCall is not a business associate at all** (§160.103) because we do not create, receive, maintain, or transmit PHI on behalf of the covered entity. When we ARE a business associate (e.g. a hosted feature that ships PHI intentionally), the BAA in `legal/BAA_DRAFT.md` applies and Exhibit A of that BAA maps every data flow explicitly.

2. Inventory of ePHI (§164.308(a)(1)(ii)(A))

Where ePHI could be created, received, maintained, or transmitted by systems in scope:

Location	Type	Retention	Encryption at rest	Encryption in transit
Local vault (<code>~/ .railcall/keys.local.json</code> , <code><station>/.railcall_workspace/keys.local.json</code>)	BYOK credentials + Ed25519 signing seed	Until user deletes	OS keychain (macOS) / DPAPI (Windows) / Secret Service (Linux) via <code>seed_store.py</code> ; degraded plaintext-file only on non-macOS if the OS backend refuses (<code>seed_store.status()</code> reports honestly)	N/A (never transmitted)
Local receipts (<code>~/ .railcall/receipts/</code>)	Signed per-action records — RailCall's OWN execution records, NOT customer clinical data	Until user deletes	0600 file permissions	N/A (never transmitted by us)
Local audit chain (<code>workbench/audit_chain.py</code> + on-disk JSONL)	Actor + event trail	Until user deletes	0600, hash-chained, Ed25519 signed	N/A
Gateway DB (<code>consumers</code> , <code>seat_reservations</code> , <code>processed_events</code> in <code>railcall-core</code> 's Postgres/SQLite)	Email, <code>sha256(api_key)</code> , seat counts, install pubkeys, Stripe customer ids	Until account deletion	Encrypted-at-rest per Render's managed Postgres (their responsibility surface, cited from Render docs)	TLS 1.3 to gateway
Gateway logs (Render service logs)	Same as DB writes + operational logs. Email is redacted from audit log per P0-4 remediation.	~30 days on Render	Render-controlled	Render-controlled

Where ePHI does NOT go, by design: - Never into any receipt body signed by us — enforced by `phi_guard.py` PHI detection + `approval_airlock.py:redact()` (see §5 for the closed egress path) - Never into the gateway request body — `/v1/seat/checkin` and `/v1/meter` both reject bodies containing anything except the three declared fields; `/v1/attestation/countersign` accepts only a hash + id - Never into any observability tool RailCall the operator uses — because it never leaves the customer's box

3. Threat & vulnerability catalogue (§164.308(a)(1)(ii)(A) sub-a)

Structured as: threat → attack path → what today's controls do → residual risk.

3.1 Local file-system exposure (P0-1)

- **Threat:** Copying, backing up, or synchronising `keys.local.json` to a location the user does not intend (Time Machine, iCloud, Dropbox, `git add`).
- **Attack path:** The signing seed's presence in that file lets any reader forge Ed25519 receipts that verify cleanly against the install's published pubkey, undermining every downstream trust claim.
- **Control (implemented):** `seed_store.py` moves the seed into the OS keychain on boot (macOS `security(1)` login keychain; Windows DPAPI; Linux libsecret via `secret-tool`). Migration is idempotent and fail-safe (keychain-write must succeed BEFORE the plaintext copy is removed — `test_seed_store.py:5b` asserts this).

- **Verification:** `test_seed_store.py` (16 checks, all pass, on both engine and CLI halves).
- **Residual risk:** The keychain protects against copying/leakage, NOT against malware running as the same OS user. Documented in `seed_store.py:_BACKEND_NOTE` and stated in the tool's own status output. Mitigation requires hardware attestation, which is customer-side.

3.2 Audit-log tamper (P0-2)

- **Threat:** A local process rewrites `audit_log.jsonl` / `integration_audit.jsonl` to remove evidence of an incident.
- **Attack path:** Post-incident inspection reads a tampered log and reports "no anomalies," meeting §164.308(a)(1)(ii)(D) audit requirement in name but not in substance.
- **Control (implemented):** `audit_chain.py` (in `railcall-engine/workbench/`) gives every audit record a `prev_hash` linking it to the previous record, Ed25519-signs the record with the install key, and adds an `actor` field. Both `audit_log()` and `integration_audit()` in `studio_server.py` route through the chain with a plain-append fallback (losing a record is worse than losing a chain).
- **Verification:** `test_audit_chain.py` (17 checks, all pass), and test 4a specifically asserts the LIMIT: tail-truncation by the install-key holder stays undetectable (chain "intact", count under-reports). `verify()` returns that caveat inside its own result so callers can't accidentally over-claim.
- **Residual risk:** Tail-truncation by the key-holder. Mitigation: off-box witness (periodic head anchoring to a service the operator does not control). This is on the roadmap — tracked as Phase-1 item 1.13 in `claude-context.md`.

3.3 PHI egress via the "PII firewall" (P0-3)

- **Threat:** A workflow field labelled generically (e.g. `notes`, `field1`) contains clinical narrative — patient name in free text, diagnosis, MRN.
- **Attack path (pre-remediation):** The pre-P0-3 `redact()` only matched field names against `SECRET_HINT = ("token", "key", "secret", "password", "apikey", "authorization", "bearer", "dsn")`. A field named `notes` passed through verbatim into the receipt payload and — if the workflow called a cloud model — into that model's request body. Verified by execution: SSN survived redaction unchanged.
- **Control (implemented):** `phi_guard.py` (in `railcall-engine/workbench/`) detects HIPAA identifiers by both field-name and by value. `approval_airlock.py:redact()` now runs three passes in order: credentials by field name → identifiers by field name → identifiers by value (recurring into nested dicts/lists). Auto-escalation: if any field name is a clinical signal (`phi_guard.CLINICAL_SIGNAL`), the whole payload escalates and unrecognised free text is redacted wholesale.
- **Install-level opt-in (P0-3 completion):** `RAILCALL_PHI_MODE` env var (or `<ws>/phi_mode.json`) declares a whole install PHI-bearing so narrative in ANY generic field is redacted, even without a clinical field name present. Verified: `test_phi_guard.py` 29 checks (was 19 pre-completion), all pass.
- **Verification:** Reproduced end-to-end via the real `airlock_card()` function against `Jane Doe / 1984-03-12 / MRN-889231 / adenocarcinoma / metoprolol / 123-45-6789` payload — none appear anywhere in the card. `api_key` still redacts to `.....`.
- **Residual risk (documented, NOT hidden):**
 - Personal names in free text are not detectable — only escalated-mode wholesale redaction handles them. Test 7a asserts this openly.
 - Biometrics, photographs, and §164.514(b)(2)(R) "any other unique identifying number" are not detectable at all.
 - Escalation keys off field NAMES. A clinical payload whose fields are all generically named (`field1`, `data`, `text`) will NOT escalate unless the install-level PHI mode is on. Callers handling PHI must set the mode.
 - This is redaction, not de-identification. It does NOT produce Safe Harbor de-identified output.

3.4 Plaintext PII in audit records (P0-4)

- **Threat:** Email addresses land in `audit_log.jsonl` in the clear. Verified pre-remediation: 2 of 49 real records contained a plaintext `email` field.
- **Attack path:** In a healthcare context, an email in an audit record is identifying information under HIPAA. Contradicted our own "never logs secret values" guarantee (which covered credentials only).

- **Control (implemented):** `email` added to the sensitive-field list; audit records no longer carry plaintext addresses. Closed in the same commit as the audit-chain (P0-2 remediation, commit `0ac6002b0`).
- **Verification:** Post-fix inspection of the same log corpus → zero plaintext email fields.

3.5 CLI signing seed at rest (P0-1 CLI half — remediated 2026-07-21)

- **Threat:** Same as §3.1 above but for a separate seed file: `~/.railcall/keys.local.json` (used by `receipt_signer.py` via the daemon). Was plaintext 0600 hex until 2026-07-21 despite the station-side P0-1 fix landing in June.
- **Attack path:** Anyone who could read the file could forge the receipts that `railcall demo` produces — the ones users show people.
- **Control (implemented, 2026-07-21):** `railcall-core/seed_store.py` (ported verbatim from engine), routed from `railcall_companion_daemon.py:_ensure_signing_seed()` and `railcall_cli.py:_persist_signing_seed()`. Live migration on the author's workstation proved seed bytes byte-identical after migration, pubkey unchanged, plaintext removed from vault.
- **Verification:** `railcall-core/tests/test_cli_seed_store.py` (8 checks, all pass). Commit `f81e11c`. Ships in station-v0.16.
- **Residual risk:** Same as §3.1 — malware running as the same user can still reach the keychain. Documented in the module and surfaced via `railcall doctor` output (`signing seed at rest: macos_keychain - ...`).

3.6 Paid-tier billing integrity

- **Threat:** A customer who cancels their subscription keeps consuming seats indefinitely because the DB row was never updated.
- **Control (implemented, 2026-07-21):** `cloud_gateway.py` now handles `customer.subscription.deleted` and `customer.subscription.updated` (terminal statuses only). Idempotent on `event_id`. Deactivated accounts can't mint (test 5 asserts this).
- **Verification:** `test_stripe_lifecycle.py` (11 checks, all pass).

3.7 Cross-install entitlement replay

- **Threat:** A paying customer copies their entitlement token to another machine to bypass their seat cap.
- **Control (implemented):** `entitlement.py` binds `install_pubkey` inside the signed body. `verify_entitlement()` checks the token against THIS install's pubkey and returns free-tier on mismatch. A copied token activates NOTHING elsewhere. Additionally, `/v1/seat/checkin` enforces the seat cap server-side against the CURRENT set of active installs.
- **Verification:** `test_activate_e2e.py` step 3a (SAME entitlement, other install → free), and `test_paid_full_e2e.py` E1-E4 (4th install refused with honest posture, nonce not burned so retry-after-free works).

3.8 Loopback-only bind, dual-control approval

- **Threat:** Cross-origin browser attack, remote takeover of the local studio, self-approval by a compromised browser.
- **Control (implemented):** `studio_server.py` binds `127.0.0.1` only (never `0.0.0.0`); a per-process CSRF session token is embedded in the served page and rate-limited; the "approve code" is NEVER templated into any served page — it prints to the launching terminal only, so a compromised browser cannot self-approve external sends or policy changes.
- **Verification:** Read `studio_server.py:12,63,71–88,104–120`. This is architectural, not a specific test — the constant nature of the loopback bind + the terminal-only approve token IS the control.

4. §164.312 technical safeguards inventory

Status vocabulary: **IMPLEMENTED** (control exists and is default on) · **PARTIAL** (exists but not default / incomplete) · **GAP** (not implemented).

4.1 §164.312(a)(1) Access Control

Spec	Status	Evidence
Unique user identification (R)	GAP	Single-user local model. No per-user identity in the station. <code>studio_server.py</code> binds loopback and treats any local caller as the operator. Mitigation for multi-user hosts is on the roadmap; single-workstation shape is defensible today for the current customer profile.
Emergency access procedure (R)	GAP	Not implemented. No documented break-glass path. Mitigation: customer's own §164.308(a)(6) incident response covers this at the operational layer.
Automatic logoff (A)	GAP	No idle timeout on the studio session. <code>SESSION_TOKEN</code> lives for the process lifetime (<code>studio_server.py:68,107</code>). Addressable spec; the alternative implemented is the loopback-only bind + short-lived process model.
Encryption/decryption (A)	IMPLEMENTED	AES-256-GCM + scrypt vault (<code>railcall-contrib/vault/vault.py:38-44,95-110</code>) is opt-in per-integration. Signing seed at rest is protected by <code>seed_store.py</code> (§3.1, §3.5).

4.2 §164.312(b) Audit Controls

Spec	Status	Evidence
Record & examine activity (R)	IMPLEMENTED	Two append-only JSONL logs (<code>integration_audit.jsonl</code> , <code>audit_log.jsonl</code>) written via <code>audit_chain.py</code> — hash-chained, Ed25519-signed, actor-tagged. See §3.2.

Residual: tail-truncation by the install-key holder is not locally detectable. Off-box witness is the mitigation, not yet implemented.

4.3 §164.312(c)(1) Integrity

Spec	Status	Evidence
Authenticate ePHI (A)	IMPLEMENTED for records	Ed25519-signed receipts (<code>workbench/railcall_signing.py</code>), independent re-verifying auditor (<code>workbench/audit_workflow.py:1-20</code>), hash-chained meter (<code>primitives/usage_meter.py</code>), issuer pubkey pinned in <code>entitlement.py:49</code> and served at gateway <code>/v1/issuer/pubkey</code> .

Caveat: this authenticates RailCall's OWN execution records. It is not a control over customer ePHI, which RailCall never stores. The BAA (Exhibit B item 2) states this explicitly.

4.4 §164.312(d) Person or Entity Authentication

Spec	Status	Evidence
Verify identity (R)	PARTIAL	Strong for AGENTS: mandatory Ed25519 signature verification on every request; no passwords or API keys carried across the wire on the paid path (blind hash for seat checkin); failed verify → 401 (<code>primitives/gateway_auth.py:1-30</code>).

Gap: no HUMAN authentication. Any local process reaching loopback is trusted as the operator. Defensible on a single-user workstation; not defensible for shared or multi-user hosts, which is exactly the healthcare deployment shape. Multi-user hardening tracked as roadmap.

4.5 §164.312(e)(1) Transmission Security

Spec	Status	Evidence
Integrity controls (A)	IMPLEMENTED	Payload hashing + idempotency keys (<code>approval_airlock.py:40-45</code>); signed receipts over every external touch.
Encryption in transit (A)	IMPLEMENTED	HTTPS to the gateway (<code>RAILCALL_GATEWAY_URL</code> → <code>https://...</code>); loopback traffic never leaves the host.
PHI scrubbing before egress	IMPLEMENTED	See §3.3 (P0-3 remediation), §3.4 (P0-4 remediation).

5. What could still fail — residual risks by category

Each risk here is a real one. Each has a stated mitigation OR a stated "documented and accepted" line. This section is what the SRA is FOR — the honest catalogue.

5.1 Same-user malware on the customer's box

The seed_store keychain protection assumes the OS's user boundary holds. A process running as the same user (compromised browser, malicious pip package, etc.) can request the keychain entry and receive it. This is a generic problem for every unattended-signing local tool.

Mitigation options if the customer's threat model requires it: - Hardware token (e.g. YubiKey) instead of keychain — breaks unattended operation. - Per-signature passphrase prompt — breaks unattended operation. Neither is implemented. Documented; accepted as inherent to unattended local signing.

5.2 Tail-truncation of the audit chain by the install-key holder

`test_audit_chain.py:4a` asserts this limit specifically. Mitigation is an off-box witness that periodically anchors the chain head to a service the operator does not control. Not implemented. On the roadmap.

5.3 Contract-violation PHI egress via `/v1/attestation/countersign`

The endpoint's contract is: "we accept only an integrity HASH and an attestation id." Enforced by explicit param typing in `cloud_gateway.py` — the endpoint has no field for "receipt body." A caller could still POST extra fields, but the handler ignores them (FastAPI's `Form(...)` typing does not forward unknown fields to Stripe or to the DB write).

Residual: a future edit that starts accepting a body field could regress this. A code-review rule ("no PHI-shaped fields on gateway request bodies") is the process control; a static scan is the roadmap for automated enforcement.

5.4 Third-party outages / secret loss

- Render is our hosting provider. An outage means the gateway is unavailable — free-tier installs keep working (local engine has no dependency on the gateway); paid installs cannot mint new entitlements but existing entitlements still verify until expiry.
- Stripe outage → new subscriptions can't be created; existing subscriptions continue to be charged by Stripe independently of our service.
- **Issuer seed loss** = catastrophic. Every outstanding entitlement becomes unforgeable-but-unreplaceable; the fix is a full station re-release with a new pinned pubkey. Backup lives in Sami's password manager + offline copy per the ceremony documented in `tools/mint_issuer_keypair.py`.

5.5 Legacy fields on the paid path

`consumers.free_runs_remaining` / `runs_used` are metered-era fields kept for free-tier quota. Paid-tier rows leave them at 0 by convention (enforced in `CONSUMER_SEAT_UPSERT`). A future edit that starts writing non-zero values on a paid row would confuse the metering read path. Schema comment on `CREATE TABLE consumers` documents this invariant.

6. Administrative safeguards (§164.308) — small-team honest posture

A one-person / two-person operation cannot map every §164.308 spec to a formal document. What we DO have:

Spec	Status	How
§164.308(a)(1)(i) Security management process	PARTIAL	This SRA is the artifact. Reviewed at each station release.
§164.308(a)(1)(ii)(A) Risk analysis	IMPLEMENTED	This document.
§164.308(a)(1)(ii)(B) Risk management	PARTIAL	Each identified gap has an owner (Sami) and a mitigation path. No formal risk register beyond this file.
§164.308(a)(1)(ii)(D) Information system activity review	IMPLEMENTED at customer install; PARTIAL at RailCall the operator	Customer: <code>audit_chain</code> (§4.2). RailCall side: Render logs, reviewed on-demand.
§164.308(a)(2) Assigned security responsibility	IMPLEMENTED	Sami Ben Chaalia is the security officer. Contact: <code>security@railcall.ai</code> .
§164.308(a)(3-5) Workforce, training, incident procedures	PARTIAL	Two-person team; onboarding/training are informal but documented in this repo's CLAUDE.md and internal notes. Formal training program is on the roadmap when headcount grows.
§164.308(a)(6) Security incident procedures	PARTIAL	Incident response for a hosted-service outage: monitor Render, page Sami. For a customer-side incident: customer's own §164.308(a)(6) applies.
§164.308(a)(7) Contingency plan	PARTIAL	Gateway state is in Render's managed Postgres (their backup surface); station code is versioned in Git + published on GitHub Releases; issuer seed is backed up offline. No formal DR drill.
§164.308(a)(8) Evaluation	IMPLEMENTED at this document	This SRA is the periodic evaluation.

7. Physical safeguards (§164.310)

Not applicable at scale — RailCall does not operate physical infrastructure. The gateway runs on Render; Render's own physical safeguards documentation is the substantive answer here and is referenced by the BAA (Exhibit A, subcontractors row).

8. Adoption

ADOPTED 2026-07-21 by:

- [x] Sami Ben Chaalia (Security Officer) — date: 2026-07-21

Adoption confirms: Sami has read this document, agrees the controls listed as IMPLEMENTED are accurately implemented, agrees the gaps listed are the real gaps, and commits to re-verifying at each station release + at least annually.

Adoption does NOT mean: - RailCall is HIPAA-compliant. HIPAA has no certifying body; compliance is self-determined and enforced per-incident by OCR. - This SRA has been reviewed by legal counsel. Counsel review is required before this document is presented to a covered entity as evidence. - The BAA is executable. `Legal/BAA_DRAFT.md` remains draft-only until counsel signs off.

Next review: at the next station release, and no later than 2027-07-21.

9. Revision history

Version	Date	Author	Change
v1	2026-07-21	Claude (drafted from Phase 1 evidence + P0-1 CLI remediation + paid-tier launch)	Initial draft against <code>railcall-engine@9521956fd</code> , <code>railcall-core@06bdd1449e</code> .

Access Control Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6 (Logical and Physical Access), HIPAA §164.312(a) **Companion evidence:** HIPAA SRA v1 in `compliance/HIPAA_SRA_v1_2026-07-21.md` (railcall-core repo).

1. Purpose

State how RailCall controls who can access RailCall systems and the customer data those systems touch. Written to match the reality of the product today — gaps are called out as gaps, controls that DO exist cite the code that implements them.

2. Scope

This policy applies to:

- **RailCall the operator** (Sami + Pat) — access to the gateway (`railcall-core.onrender.com`), Render dashboard, Stripe dashboard, Postgres, GitHub repositories, and the issuer signing seed.
- **The local RailCall station** deployed on a customer machine — access to the vault, the workflow-editing studio, and the signing seed.

Physical safeguards and customer-side IAM are OUT of scope — those live with the customer or with our hosting providers (Render, Cloudflare).

3. Principles

- **Least privilege.** Every credential — human, agent, or service — receives only the specific scope it needs. Restricted API keys are preferred over full-access ones (the Stripe restricted-key ceremony in the July 2026 paid-tier launch is the canonical example).
- **Local-first by construction.** The station binds `127.0.0.1` only, never `0.0.0.0` (`studio_server.py`). This IS an access control — it removes the entire network attack surface for the studio.
- **Dual control for irreversible actions.** The studio approval token is printed to the launching terminal only, never templated into any served page (`studio_server.py:71-88`). A compromised browser cannot self-approve.
- **Fail closed.** Every gate refuses on unknown input rather than allowing by default. Blind auth on `/v1/seat/checkin` returns 401 on unknown hash; issuer mint returns 503 if the seed is unset; entitlement verification returns free-tier on any tamper.

4. Controls in force today

4.1 §164.312(a)(2)(iv) Encryption of at-rest secrets — IMPLEMENTED

The Ed25519 signing seed for both the station and the CLI is held in the OS keychain — macOS login keychain via `security(1)`, Windows DPAPI, Linux Secret Service via `libsecret`. Implementation in `seed_store.py`; migration from historical plaintext-file storage is idempotent and fail-safe (the keychain write MUST succeed before the plaintext copy is dropped — `test_seed_store.py:5b` guards this).

BYOK credentials for integrations sit in a 0600 vault, AES-256-GCM + scrypt encrypted, opt-in per integration (`vault.py:38-44,95-110`).

Honest limit: the keychain protects against copying, backups, and sync-service leakage. It does NOT stop malware running as the same OS user. Documented in `seed_store.py` `_BACKEND_NOTE` and surfaced via `railcall doctor`.

4.2 §164.312(d) Person/entity authentication — PARTIAL (strong for agents)

- **Agent-to-gateway auth is strong:** mandatory Ed25519 signature verification on every request; no passwords carried across the wire on the paid path (blind sha256(api_key) for `/v1/seat/checkin`); failed verify returns 401

(`primitives/gateway_auth.py:1-30`).

- **Cross-install entitlement replay is prevented:** entitlement tokens are bound to the install pubkey inside the signed body; a copied token activates NOTHING on another machine (`entitlement.py verify_entitlement`).
- **Human authentication on the station itself is a GAP** — see §5.

5. Accepted gaps

5.1 §164.312(a)(2)(i) Unique user identification — GAP

The station is a single-user local runtime. There is no per-user identity; any local process reaching loopback is trusted as the operator. Multi-user hardening is on the product roadmap. For today, mitigation is the customer OS user boundary — a customer deploying to a shared host must rely on their own login controls.

5.2 §164.312(a)(2)(ii) Emergency access procedure — GAP

No break-glass procedure documented in the station. The customer §164.308(a)(6) incident response covers this at the operational layer.

5.3 §164.312(a)(2)(iii) Automatic logoff — GAP

No idle timeout on the studio session. The compensating control is the loopback-only bind — an attacker with local session access on the customer box has broader problems than an unlocked studio. Timeout addition is on the roadmap.

6. Access to RailCall-operator systems

System	Access holders	Grant	Revoke
Render (gateway)	Sami, Pat	Direct env-var edits on the railcall-core service; env groups require explicit service linking	Remove from Render team
Stripe	Sami (admin invited by Pat 2026-07-21), Pat (owner)	Standard keys via account owner; restricted keys per-task	Delete from Stripe team; revoke keys individually
GitHub (patl4588 org)	Sami, Pat	Repo collaborator invite	Remove collaborator
VPS 157.230.177.45 (railcall.ai website)	Sami (sami@ user, <code>~/.ssh/id_ed255199</code>), Pat (root)	SSH key add	Remove authorized_keys entry
Issuer signing seed (paid-tier root)	Sami only	1Password + offline backup	If lost: re-mint keypair, re-pin <code>ISSUER_PUBKEY_HEX</code> , cut new station release

7. Access review

Reviewed at each station release AND at least annually. Any change to the "access holders" column requires a corresponding update here.

8. Related controls

- Encryption of at-rest secrets: measure `P0-1` (`seed_store` | integration)
- Audit trail: measure `P0-2` (`audit_chain`), enforces §164.312(b)
- PHI egress: measure `P0-3` (`phi_guard`), enforces §164.312(e)(1) transmission security

Encryption Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.1/CC6.7 · HIPAA §164.312(a)(2)(iv) + §164.312(e)(1)(A/B) · NIST SP 800-52r2 (TLS) · NIST SP 800-175B (algorithms) **Companion evidence:** HIPAA SRA v1 (`compliance/HIPAA_SRA_v1_2026-07-21.md` , `railcall-core`), Access Control Policy v1.0 ADOPTED (Probo).

1. Purpose

State the algorithms, key management, and boundary conditions under which RailCall encrypts data at rest, in transit, and in use. Written to match the reality of the product today — every algorithm cited is one the code actually uses, every key lifecycle described is one an operator can reproduce.

2. Scope

- Every path where RailCall touches customer credentials, receipts, entitlements, or PHI (which the product actively refuses to store — see §5).
- The signing seed that anchors receipt integrity and paid-tier entitlements — the SoT for downstream trust claims.
- The vault holding BYOK integration credentials.
- The gateway transport (`railcall-core.onrender.com`) and Postgres.

Encryption inside third-party providers we depend on (Render's managed Postgres, Stripe, Anthropic) is called out where it applies but is ultimately their responsibility surface — cross-referenced by SLA rather than reimplemented.

3. Algorithms in use

Purpose	Algorithm	Where
Receipt + entitlement signatures	Ed25519 (RFC 8032)	<code>railcall_signing.py</code> , <code>receipt_signer.py</code> , <code>entitlement.py</code>
Signature fast path	<code>cryptography</code> package native Ed25519	<code>railcall_signing.py:sign_raw</code> — falls back to <code>ed25519_pure</code> when unavailable, byte-identical output per RFC determinism
Vault at rest (BYOK creds)	AES-256-GCM with a script-derived key	<code>railcall-contrib/vault/vault.py:38-44,95-110</code>
Payload integrity hashing	SHA-256 (canonical JSON, sorted keys)	<code>receipt_signer.py:canonical_bytes</code> , <code>approval_airlock.py:40-45</code>
Audit chain linking	SHA-256 (<code>prev_hash</code> per record)	<code>workbench/audit_chain.py</code>
Blind seat auth over the wire	SHA-256(api_key) → hex	<code>cloud_gateway.py</code> <code>/v1/seat/checkin</code> , <code>/v1/meter</code>
Transport	TLS 1.3 to <code>railcall-core.onrender.com</code> (Render-terminated)	<code>RAILCALL_GATEWAY_URL</code> set to <code>https://</code> in the CLI; loopback traffic on the station never leaves the host
OS keystore-backed seed protection	macOS Keychain (Data Protection API) / Windows DPAPI / Linux Secret Service	<code>seed_store.py</code>

Algorithms deliberately NOT used (worth stating so a review can't misread the absence):

- **RSA:** not used for RailCall-generated keys. The only RSA presence is Probo's OAuth2 signing key in the dev infra tree, out of RailCall scope.
- **SHA-1 / MD5:** not used anywhere in RailCall code paths.
- **Symmetric key exchange in the field:** not used — we do not run a session-key protocol; every symmetric encryption is a local at-rest concern.

4. Key management

4.1 The install signing seed (paid-tier root of receipt trust)

- **Generation:** 32 bytes from `os.urandom` (CSPRNG) at first station boot.
- **Storage:** OS keychain via `seed_store.py`. macOS `security(1)` login keychain (secret piped over stdin, never argv). Windows DPAPI encrypts the seed with the current user's credentials and writes the ciphertext beside the vault. Linux libsecret via `secret-tool`.
- **Fallback:** if no keystore backend is available (e.g., Linux headless without libsecret), the seed stays in the 0600 vault file. `seed_store.status()` reports `at_rest="plaintext_file"` — the honest posture, never a false claim of protection.
- **Migration:** on every boot, `seed_store.migrate()` idempotently moves any pre-existing plaintext seed into the keystore. **Ordering is a safety property:** the keystore write must succeed BEFORE the plaintext copy is dropped, or a failed migration would orphan the install identity. `test_seed_store.py:5b` asserts this.
- **Rotation:** via `railcall rotate-key`. Publishes the new pubkey, archives the previous public doc for verifier continuity, writes the new seed through `seed_store` in the same place `_ensure_signing_seed` reads. Rotation is user-initiated only; there is no automatic rotation.
- **Never leaves the box:** the private seed is never displayed, logged, or transmitted. `install.sh` explicitly excludes `keys.local.json` from the file set it distributes.

4.2 The issuer seed (paid-tier authority)

- **Generation:** via `tools/mint_issuer_keypair.py` — one-shot ceremony, refuses to silently discard the seed.
- **Storage:** environment variable `RAILCALL_ISSUER_SEED` on the Render gateway service. Set service-level, NOT via env groups (an env group must be explicitly linked to a service before values flow; direct service-level env vars trigger a redeploy on save).
- **Backup:** 1Password + offline copy. Losing the seed is catastrophic (see §7).
- **Rotation:** any rotation invalidates every outstanding entitlement. Documented as a deliberate, human-only operation — no automation.
- **Reachability:** `/v1/issuer/pubkey` (public) returns the derived pubkey; the seed itself never appears in any API response, never in any log line, never in any error message. Missing seed → endpoints fail closed with 503 (`cloud_gateway.py:_issuer_seed`).

4.3 The BYOK vault (integration credentials)

- **Encryption at rest:** AES-256-GCM + scrypt KDF, opt-in per integration.
- **File permissions:** 0600 in all cases, unencrypted or encrypted; pinned at fd creation (`vault_io.py` — `os.fchmod` before any secret bytes land).
- **Atomicity:** every write is temp → fsync → `os.replace` → chmod → dir fsync. A crash never leaves a truncated vault; a partially-written file cannot masquerade as an empty vault (`vault_io.load()` raises `VaultCorruptError` rather than silently returning `{}`).
- **Reserved keys:** `_railcall_signing_seed` is a reserved vault key filtered out of every integration listing (`railcall_signing.is_reserved_vault_key`).

4.4 Session tokens (studio access)

- Per-process CSRF token (32 bytes hex), embedded in the served page. Lives for the process lifetime, discarded on studio exit.
- Loopback-only bind (`127.0.0.1`) is the primary access boundary; the token is a secondary defence against cross-origin fetches.

- The "approve code" for irreversible actions is printed to the launching terminal only — never templated into any served page — so a compromised browser cannot self-approve. See Access Control Policy §3.

4.5 Stripe API keys

- Live keys held in Render env vars, `sync: false` (per-environment, never in the repo).
- Restricted keys preferred over standard keys for any scoped operation (Products+Prices write for price setup, etc.). Restricted keys are treated as single-use and revoked after the task that needs them.

5. What we do NOT encrypt, and why

- **Customer workflow inputs and outputs:** the product does not store these. They live on the customer's own machine in files the customer controls. Encryption of that data is the customer's concern, not ours. This is not a gap — it is architectural. Stated in the SRA §1 ("Key architectural fact"), stated in the BAA Exhibit A (data flows).
- **PHI in receipt bodies:** does not appear, by construction. `phi_guard.py` + `approval_airlock.py:redact()` remove HIPAA identifiers before the receipt is signed — three-pass (credentials by field name → identifiers by field name → identifiers by value), with auto-escalation on clinical field-name signals. `test_phi_guard.py` 29/29. This is a stronger property than encryption at rest — the sensitive data never reaches the write path in the first place.
- **PHI in gateway requests:** does not appear, by contract. `/v1/seat/checkin` and `/v1/meter` accept a fixed set of scalar fields (blind key hash + install pubkey + nonce, or key hash + run count + nonce); `/v1/attestation/countersign` accepts an integrity hash and an attestation id. No endpoint accepts a receipt body or payload. Contract-violation risk documented in SRA §5.3.
- **Free-tier metered columns in the gateway DB:** `consumers.free_runs_remaining` / `runs_used` are integers, not sensitive. Not encrypted at the column level; protected by DB-level access via Render.

6. Transport

- **CLI → gateway:** HTTPS to `https://railcall-core.onrender.com`, TLS 1.3 negotiated with Render's certificates. No HTTP fallback.
- **Station → gateway:** same.
- **Browser → studio:** loopback only (`http://127.0.0.1:...`), no TLS. Justified by the loopback bind — the traffic physically cannot leave the host, so TLS would add complexity without adding protection.
- **Station internal (workflow engine → integrations):** governed by the customer's own configuration. RailCall's role is redaction before egress (§5), not termination-point encryption.

7. Loss / compromise procedure

Written per-key so an operator under stress has a script to follow, not prose to parse.

Install signing seed lost (single install): 1. That install can no longer sign new receipts. Existing receipts remain verifiable because the pubkey is published (`signing_pubkey.json`). 2. Run `railcall rotate-key` — mints a fresh keypair, archives the old public doc to `signing_pubkey.prev-<ts>.json`, publishes the new one. 3. Pre-rotation receipts still verify via `verify --key <archived>`.

Install signing seed suspected COMPROMISED: 1. Same as loss, PLUS: notify anyone who trusts receipts from that install that receipts dated between compromise-suspected and rotation should be treated as unverified. 2. If unclear whether receipts were forged post-compromise, treat the whole install as compromised — wipe, reinstall, generate a fresh identity.

Issuer seed lost: 1. Every outstanding entitlement becomes unreplaceable. Existing installs continue to work with the entitlement they already hold until it expires. 2. Recovery: `tools/mint_issuer_keypair.py mint --seed-out <path>`, back up the new seed, edit `ISSUER_PUBKEY_HEX` in `railcall-engine/workbench/primitives/entitlement.py`, cut a new station release (see `RELEASE_station-v0.16.md` for the ceremony), re-pin `install.sh` + `install.ps1`, deploy the mirror, set the new seed as `RAILCALL_ISSUER_SEED` on Render. 3. Every existing paying customer must re-run the installer to get a station that trusts the new key. Communicate this via email + release notes before deploying.

Issuer seed suspected COMPROMISED: 1. Same as loss, PLUS: rotate immediately (do not wait to build the new station release — a compromised seed lets an attacker mint valid-looking entitlements today). 2. Every entitlement minted since compromise-suspected is

untrusted. Communicate the rotation to affected customers and re-mint their entitlements against the new authority.

BYOK vault entry compromised: 1. Customer's concern (the credentials are theirs). Actionable steps: rotate the compromised credential in the upstream service, remove the entry from `keys.local.json`, re-authorize. 2. RailCall the operator has no visibility into which BYOK credentials exist — nothing to notify.

8. Third-party encryption we rely on

Provider	What we entrust	Their responsibility
Render	Gateway Postgres, service logs, TLS termination	Encryption at rest (managed Postgres), TLS 1.3 in transit, secret storage for <code>sync:false</code> env vars
Stripe	Payment card numbers, subscription state	PCI-DSS Level 1; RailCall never sees full card numbers
Anthropic (Probo LLM only)	Prompt text sent from Probo's internal agents	Anthropic API contract — no customer PHI reaches this path because Probo is an internal tool, not customer-facing
GitHub	Source code, release artifacts, station tarball	Standard GitHub SLA; no secrets in the repo (verified by the release script's leak gate)
VPS 157.230.177.45	railcall.ai static site + installer mirror	SSH-key-authenticated access only; no customer data on the box

9. Review

Reviewed at each station release AND at least annually. Algorithm choices are re-reviewed against NIST SP 800-131A (retired algorithms) at each annual review. Any change to §3 (algorithms) requires a corresponding change here BEFORE the code lands.

10. Related controls

- Signing seed at rest: measure `P0-1` (`seed_store` integration)
- Audit trail integrity: measure `P0-2` (`audit_chain`), enforces §164.312(b)
- PHI egress prevention: measure `P0-3` (`phi_guard`), enforces §164.312(e)(1)
- Person authentication (agent-side): Access Control Policy §4.2
- Loss-of-trust procedures: this policy §7

Incident Response Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC7.3/CC7.4/CC7.5 · HIPAA §164.308(a)(6) · GDPR Art. 33 (breach notification) **Companion documents:** Access Control Policy · Encryption Policy · HIPAA SRA v1 · BAA_DRAFT.md §3 (breach reporting).

1. Purpose

Give an operator under stress a script to follow — detection, triage, containment, notification, recovery, post-mortem — not prose to parse. Every window in this document is one we can actually meet with the team of two we have today.

2. Scope

- Security incidents on RailCall-operated infrastructure: gateway (`railcall-core.onrender.com`), Render Postgres, VPS `157.230.177.45` (railcall.ai + installer mirror), the issuer signing seed, Stripe account, GitHub organisation.
- Product-level incidents affecting shipped stations: forged receipts, entitlement compromise, integrity failures.
- Third-party incidents that reach us: Render outage, Stripe outage, Anthropic outage, GitHub compromise, dependency supply-chain compromise.

OUT of scope: incidents entirely on the customer's own machine that never touch our infrastructure or invalidate a RailCall-signed artifact. Those belong to the customer's own §164.308(a)(6) program; we assist under the BAA when we're a business associate for that deployment.

3. Severity classification

Severity	Definition	Response window (acknowledge / contain / notify)	Example
SEV-1 Critical	Trust root compromised OR customer data at risk OR paid tier globally unavailable	30 min / 4 h / 10 days per BAA §3.3	Issuer seed leaked; receipts being forged in the wild; gateway serving PHI
SEV-2 High	Non-root credential compromised OR partial paid- tier failure OR audit evidence integrity compromised	2 h / 24 h / per contract	Stripe key leaked; audit chain break detected; one install seed leaked
SEV-3 Medium	Degraded service, contained, no customer impact	24 h / 5 business days / discretionary	Gateway 5xx spike; Postgres slow; one webhook delivery failing
SEV-4 Low	Observed anomaly, no operational impact	Next business day / at your leisure / no	One-off failed login attempt; noisy log; false positive

Escalate up on any of: legal/regulatory exposure discovered, customer directly harmed, media attention likely, or when in doubt. The cost of over-escalating is a meeting; the cost of under-escalating is a breach that turns into willful neglect under OCR review.

4. Detection — what triggers an incident

Sources ranked by trust:

1. **Customer report** — email to `security@railcall.ai`. Treat every report as at least SEV-3 until triaged.
2. **Automated invariants failing** — see §4.1 for the specific ones.
3. **Third-party notification** — Render status page, Stripe security alert, GitHub security alert on a dependency, upstream CVE.

4. **Our own observability** — Render logs, `pm2 logs railcall-website`, `docker logs proba-probo-1`, ad-hoc `curl` health checks.
5. **Internal review** — Sami/Pat noticing something odd during normal work.

4.1 Automated invariants (checked on every relevant read)

Invariant	Where	On break
Receipt signature verifies against pinned pubkey	<code>entitlement.py verify_against_install</code>	Receipt returns <code>SIG_FAIL</code> (not <code>SIG_UNSIGNED</code> — that's honest legacy). Any <code>SIG_FAIL</code> in a customer report → SEV-1.
Audit chain <code>prev_hash</code> consecutive	<code>audit_chain.py verify()</code>	Returns <code>chain_intact=False</code> + locates the break. Customer or auditor running verify sees this. SEV-2 by default; SEV-1 if the break correlates with a known access event.
Issuer pubkey served matches station pin	<code>curl /v1/issuer/pubkey</code> returns the pinned hex	Mismatch = SEV-1 (either the seed was rotated without a station release, or the seed was replaced without our knowledge).
<code>RAILCALL_ISSUER_SEED</code> set on gateway	<code>/v1/entitlement/mint</code> returns 200 for a valid paid caller	503 = billing broken; SEV-2 until the seed is restored.
PHI redaction still enforced	<code>phi_guard</code> still importable at station boot	ImportError in station logs = SEV-2 for any install with <code>RAILCALL_PHI_MODE=on</code> ; SEV-3 otherwise.
Seat cap enforced	<code>/v1/seat/checkin</code> refuses over-capacity installs with 402	If a customer reports "I paid for 3 but 5 machines work", check <code>/v1/seat/status</code> — if it agrees, treat as SEV-2 (billing integrity).

5. Roles

Two-person team. Roles rotate by availability, not by title.

- **Incident Commander (IC):** whoever picks up first — owns the incident until handoff or closure. Their job is decisions and communication, NOT to be the one debugging (that's the technical responder). In a two-person team, one person is often both, but the ROLES are distinct so nothing gets dropped when one person is on a plane.
- **Technical Responder (TR):** the one making changes. Rotates a screen with the IC on a Signal call for anything SEV-2 or worse.
- **Security Officer (SO):** Sami. Final decision authority on customer notification, regulatory reporting, and public disclosure. Does not need to be the IC.

6. Runbook — the actual steps

6.1 Every incident (any severity)

1. **Open a shared note.** `~/incidents/YYYY-MM-DDThmm-<slug>.md`. Time-stamped, append-only, on Sami's box. This IS the incident record; do not rely on memory or chat scrollback.
2. **State the invariant that broke.** Not the symptom — the invariant. "The receipt signed by X does not verify against the pubkey pinned in station-v0.16" is an invariant. "A customer emailed us angry" is a symptom.
3. **Assign IC + TR.** In writing, in the note.
4. **Set severity from §3.** Update at any time; log every change with the reason.
5. **Take one containment action within the response window.** Even a bad containment (e.g., "took gateway offline for 5 min") is better than paralysis. Log it.
6. **Restore.** Follow §7 per-scenario runbooks.
7. **Notify per §8.**
8. **Post-mortem within 5 business days.** §9.

6.2 Notifications inside the team

- SEV-1: Signal call within 30 min. If IC can't reach TR/SO, escalate to phone.
- SEV-2: Signal message within 2 h + Signal call for anything not obvious.
- SEV-3: Signal message within 24 h.
- SEV-4: Log in the note; discuss at the next sync.

7. Scenario runbooks

Each below is a decision tree — no "figure it out" cells.

7.1 Issuer signing seed compromise or loss

Severity: SEV-1 unconditionally.

1. **Assess reach.** Has ANY receipt or entitlement been minted against this seed since compromise-suspected? If yes: every one is untrusted. If no: recovery is clean.
2. **Rotate immediately.** Do NOT wait to schedule downtime. - Run `python3 tools/mint_issuer_keypair.py mint --seed-out ~/railcall-issuer-seed-YYYYMMDD.txt` on Sami's box. - Back up the new seed to 1Password + offline paper BEFORE proceeding.
3. **Deploy the rotation.** Follow the Encryption Policy §7 issuer-seed-lost runbook (edit `entitlement.py` pin, cut station-vN+1, re-pin `install.sh` / `install.ps1`, deploy contrib mirror, `ssh sami@157.230.177.45 && cd ~/railcall-contrib && git pull && pm2 restart railcall-website`).
4. **Set new seed on Render.** Service-level env var `RAILCALL_ISSUER_SEED`. Verify: `curl https://railcall-core.onrender.com/v1/issuer/pubkey` returns the new pubkey.
5. **Communicate.** Email every paying customer: "we rotated the issuer key. Reinstall your station via `curl -fsSL https://railcall.ai/install.sh | bash`, then `railcall activate` to re-mint your entitlement." Include the reason at your discretion — if compromised, say so; the trust cost of hiding it is higher than the trust cost of saying it.

7.2 Individual install signing seed compromise

Severity: SEV-2 (single customer scope).

1. Confirm reach — is this install signing anything the customer or a downstream verifier trusts? Usually yes (their `railcall demo` receipts).
2. Instruct the customer to run `railcall rotate-key`. This mints a fresh keypair, archives the old public doc, so pre-rotation receipts stay verifiable via `verify --key <archived>`.
3. Optionally: reinstall the station cleanly to be sure nothing else is compromised.
4. If unclear whether the compromise is limited to the seed — treat as SEV-1 for the customer's own workflow data (not for RailCall as a whole).

7.3 Audit chain break

Severity: SEV-2 default; SEV-1 if correlated with a known access event.

1. Read `verify()` output — it locates the break to a specific record index.
2. Before that index, the chain is intact; after that index, it's rewritten.
3. If the customer holds their own audit logs (usual case), the customer is the SoT — we assist their investigation but do not have their evidence.
4. If the break is in a hosted-service log (`Render` logs the gateway keeps), our copy is the SoT — extract the pre-break slice, treat post-break as tampered.
5. Document the break, its context, and the recovery in the incident note. `verify()` returns the caveat text — copy it in verbatim so nobody accidentally over-claims.

7.4 Gateway outage (Render)

Severity: SEV-3 usually; SEV-2 if it lasts > 1 hour or during a paid customer's mint window.

1. Check Render status: <https://status.render.com>.
2. Free-tier installs are unaffected (they do not depend on the gateway). Communicate this on Twitter/Discord if outage is public.
3. Paid installs cannot mint NEW entitlements but existing entitlements verify locally until expiry — this is by design, not a bug.
4. If Render is up and only our service is down: check `docker logs` via the Render dashboard, roll back to the previous deploy if the current one is broken.
5. Post-outage: if any paid customer's entitlement expired during the outage, offer them an extension via email.

7.5 Stripe outage or credential compromise

Severity: SEV-2.

1. **Outage:** existing subscribers keep being charged by Stripe independently of our service. New checkout is blocked; add a banner to the pricing page pointing to Discord for updates.
2. **Credential compromise:** rotate ALL Stripe keys immediately (Standard + every Restricted). Update `STRIPE_SECRET_KEY` + `STRIPE_WEBHOOK_SECRET` on Render. Verify webhook signature verification is still passing (`POST /v1/webhooks/stripe` with unsigned body returns 400 "Unable to extract timestamp and signatures").
3. Search Stripe dashboard for any suspicious sessions/customers created during the compromise window. Refund fraudulent charges.

7.6 VPS `157.230.177.45` (railcall.ai) compromise

Severity: SEV-2.

1. `ssh sami@157.230.177.45` — check `who`, `last`, `ss -tulpn`, `pm2 logs`.
2. Rotate the SSH key (`~/.ssh/id_ed25519`) — add a new key to `authorized_keys`, remove the old, verify you can still log in with the new before disconnecting.
3. If installer or station tarball on the mirror was tampered with — `railcall-contrib/website-v2/public/install.sh` or `.tar.gz` — that's SEV-1: users installing from the mirror got a compromised binary. Byte-compare against `railcall-core/install.sh` and the GitHub release tarball. If mismatch: fix the mirror, redeploy, and email every user who installed during the compromise window telling them to reinstall.

7.7 Dependency / supply chain compromise

Severity: SEV-2 default; SEV-1 if the compromised package is on the station-signing or gateway-secret path.

1. Identify which package + version. Search `railcall-core/requirements.txt` and `railcall-engine`'s Python imports.
2. If on the signing path (`cryptography` package etc.) — treat as SEV-1: might have signed with a compromised signer. Rotate signing seeds.
3. Pin the fixed version; re-run every test suite; re-cut station release if any station-bundled dependency is affected.
4. Retrospectively verify: are any of our receipts in customers' hands signed against the compromised version? If yes, communicate.

8. External notification

- **Customers affected:** direct email from `security@railcall.ai` within the response window in §3. Template lives in `compliance/templates/breach_notice.md` (TO BE WRITTEN). Include: what happened, when, what data was affected, what we've done, what they should do, when we'll next update them.
- **Covered entities with a BAA:** per BAA §3.3, within 10 days of discovery of a breach affecting their PHI. Draft template in `legal/BAA_DRAFT.md` §3 — DO NOT SEND without counsel review the first time.
- **Regulators:**

- **HHS OCR** for HIPAA breaches affecting 500+ individuals: 60 days per § 164.408. Fewer than 500: annual summary.
- **State AGs:** varies by state; assume 30 days unless research proves otherwise for the specific state.
- **GDPR supervisory authority:** 72 hours per Art. 33 if RailCall is a processor for EU personal data.
- **Public disclosure:** SO decides. Default: proactive if any customer is affected, silent if the incident is fully contained pre-impact and disclosure serves no protective purpose. Do NOT lie by omission — if asked, answer honestly.

9. Post-incident review

Within 5 business days of closure for SEV-1/SEV-2. Simple format:

1. **What happened** (timeline in the incident note, cleaned up).
2. **Why it happened** — root cause, not proximate cause. "The pod restarted" is proximate; "we had no memory limit set, so a leak in dep X ate the pod" is root.
3. **What we did well.**
4. **What we did badly.** Blameless — the process is what failed, not the person.
5. **Action items** with owners and due dates. Each one is a real ticket, not a wishlist.

Post-mortems land in `~/incidents/post-mortems/` (Sami's box) and are shared with any customer we notified about the incident. Redact only what genuinely can't be shared (customer identity, active security-through-obscurity mitigations); everything else is public to the affected parties.

10. Testing

Once a quarter, at least one scenario from §7 is tabletop-exercised — a 30 min Signal call where IC + TR walk through the runbook against a hypothetical trigger. Log the exercise as a "SEV-4 (tabletop)" incident so the process itself gets its reps.

At least once per year, one scenario is executed end-to-end in a staging environment (or the real environment during a known low-traffic window) — issuer seed rotation is the natural annual exercise since it's the highest-blast-radius runbook.

11. Related documents

- **Access Control Policy** — who has the credentials that get compromised in §7.1, §7.5, §7.6.
 - **Encryption Policy** — the algorithms and key lifecycles referenced from §7.1, §7.2, §7.7.
 - **HIPAA SRA v1** — the threat catalog (§3) whose realizations this policy responds to.
 - **BAA_DRAFT.md** — §3.3 breach reporting window.
 - **audit_chain.py + test_audit_chain.py** — the primary detection tooling for §4.1 invariant "audit chain intact".
-

Data Retention & Deletion Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.5 / P1 (Privacy) · HIPAA §164.316(b)(2)(i) 6-year retention · GDPR Art. 5(1)(e) storage limitation, Art. 17 right to erasure **Companion documents:** Access Control Policy · Encryption Policy · Incident Response Policy · HIPAA SRA v1 · Privacy Policy (railcall-contrib/website-v2/app/legal/privacy).

1. Purpose

State, per data type, how long RailCall keeps it, why, and how to make it go away. Two separate stories: what RailCall the operator keeps in our own infrastructure, and what the shipped station keeps on the customer's box. Different owners, different rules, different rights.

2. The two stories

- **RailCall-operator side:** we control this data — gateway Postgres rows, Stripe subscription state, Render logs, Anthropic API call metadata (Probo dev only). Retention rules in §3 and §4.
- **Station side (on the customer's box):** the customer controls this — vault, receipts, audit chain, workflow inputs/outputs. Retention rules in §5. This is not our data to delete; we document what the station puts on disk and give the customer commands to remove any of it.

Cross-cutting principle: we retain the MINIMUM amount that satisfies (a) legal + regulatory requirements, (b) our own operational needs (billing integrity, incident forensics), and (c) contractual commitments to customers.

3. RailCall-operator data — retention windows

Data	Where	Retention	Justification
<code>consumers</code> row (email, api_key_hash, plan, seat_count, stripe_customer_id)	Render Postgres	Life of account + 6 years after account closure	HIPAA §164.316(b)(2)(i) requires 6 years for records related to compliance. Account closure = deletion request OR 12 months of inactivity, whichever first.
<code>seat_reservations</code> row (api_key_hash, install_pubkey_hex, first/last_seen_at)	Render Postgres	Rolling 30 days from last checkin (<code>_SEAT_TTL_DAYS</code>)	Pruned automatically by every <code>/v1/seat/checkin</code> call. No historical retention needed — enforcement is a snapshot property.
<code>processed_events</code> row (event_id, processed_at)	Render Postgres	90 days	Long enough for Stripe retries (72 h max) + our own idempotency window; short enough to keep the table bounded.
Render service logs (gateway HTTP + stdout)	Render (their infra)	Render default (~30 days)	Their retention surface. We don't extend it.
Stripe subscription + payment records	Stripe (their infra)	7 years per Stripe defaults + tax law	Stripe's responsibility surface.
Anthropic API call metadata (from Probo)	Anthropic (their infra)	Anthropic default (30 days, zero-retention on request)	Probo is dev-only — customer data does not reach this path.
Incident notes + post-mortems	Sami's box (<code>~/incidents/</code>)	6 years	Same clock as <code>consumers</code> rows — an incident affecting a customer is part of that customer's compliance record.

4. Deletion — how each of the above is removed

4.1 On account cancellation (subscription cancelled in Stripe)

Automatic: 1. Stripe fires `customer.subscription.deleted` (or `.updated` with a terminal status). 2. `cloud_gateway.py` webhook sets `consumers.status = 'inactive'` and `seat_count = 0` (cb6ab14, tested by `test_stripe_lifecycle.py`). 3. Consumer is refused for any new mint (§164.312 access control). 4. Seat reservations for that account age out under the 30-day TTL naturally.

The row itself is NOT deleted on cancellation — it stays as an `inactive` record for 6 years per §3. This matters: - **Why we keep it:** proof-of-relationship for regulator or audit inquiry (which paying customer we served, when). - **Why the customer isn't harmed:** all the sensitive material (raw `api_key`, session tokens) was already hashed or expired at earlier stages; a stale `inactive` row cannot be re-activated without a fresh Stripe subscription minting a new key.

4.2 On explicit customer erasure request (GDPR Art. 17 / "delete my account")

Customer emails `privacy@railcall.ai`. Within 30 days:

1. Verify identity via the customer's registered email + a challenge (activate a signed token from their install; only they can produce it).
2. Locate every row bound to their email + `stripe_customer_id`: `consumers`, `seat_reservations`, `processed_events` referencing their session ids, any incident notes citing them.
3. Delete the `consumers` row and `seat_reservations` rows outright. Redact `processed_events` and incident notes — replace the customer identifier with a stable pseudonym so referential integrity survives, but nothing traces back to a person.
4. Stripe: request customer deletion via Stripe dashboard — they retain payment records per tax law but detach personal data.
5. Send the customer a confirmation email with a summary of what was deleted, what was retained (Stripe payment records), and why.

Note: HIPAA §164.316(b)(2)(i) 6-year retention CAN legally override GDPR Art. 17 when the customer is a covered entity and the data relates to compliance controls we owe them. In that case: retain the minimum, redact the personal identifiers, and document the retention basis in our reply.

4.3 On issuer seed rotation

Not a deletion event, but tangential: - All entitlement rows in `consumers` remain valid until each install re-mints against the new authority. - Old `signing_pubkey.json` files stay archived on the customer's own box; pre-rotation receipts still verify against the archive (Encryption Policy §7.1).

5. Station-side data (on the customer's machine)

RailCall does not read, transmit, or delete these. We document what the station writes so the customer can control their own retention.

Data	Path	Default retention	How to remove
Vault	<code>~/.railcall/keys.local.json</code> (CLI) and <code><station>/.railcall_workspace/keys.local.json</code>	Forever until user deletes	<code>rm -P <path></code> (macOS overwrite-then-unlink)
Signing seed	OS keychain (macOS/Windows/Linux) — see Encryption Policy §4.1	Forever until user deletes	<code>security delete-generic-password -s ai.railcall.signing-seed</code> on macOS; equivalent on other OSes
Receipts	<code>~/.railcall/receipts/**</code>	Forever until user deletes	<code>rm -rf ~/.railcall/receipts</code>
Audit chain	<code><station>/audit_log.jsonl</code> , <code><station>/integration_audit.jsonl</code>	Forever until user deletes	<code>rm <path></code> — note this ALSO destroys the chain-based tamper evidence for pre-deletion events
Workflow inputs / outputs	Wherever the customer's workflow writes them	Under the customer's own retention policy	Customer's responsibility
Studio session token	Memory only — process lifetime	Killed on <code>railcall studio</code> exit	Kill the process
Entitlement	<code><station>/.railcall_workspace/entitlement.json</code>	Until token expiry or user deletes	<code>rm <path></code> ; station reverts to free tier on next check

Customer-side uninstall: `rm -rf ~/.railcall/` removes everything except OS-keychain entries. To remove those too: the `security` command above (macOS) or platform equivalent.

6. HIPAA §164.316(b)(2)(i) — the 6-year retention story

HIPAA requires policies + procedures + documentation of compliance-related decisions be retained for 6 years from the later of "creation" or "last effective date." What that means for us:

- **This policy** and every other Probo-tracked policy: retained in Git indefinitely (Git IS the retention layer); a specific version is retained for 6 years from the moment a subsequent version supersedes it.
- **Adopted SRA versions** (`compliance/HIPAA_SRA_v1_2026-07-21.md`): 6 years from adoption date, minimum.
- **Audit records** (incident notes, post-mortems, customer breach notifications): 6 years from the incident close date, minimum.
- **Access review records** (see Access Control Policy §7): 6 years from the review date.
- **Consumer records** (as above): 6 years from account closure.

These are minimums under HIPAA. GDPR erasure requests may reduce these for personal data specifically — see §4.2 for the balancing rule.

7. Backup + recovery windows

- **Render Postgres:** Render's managed backup surface — see their SLA. We do not maintain a separate backup.
- **VPS** `157.230.177.45` : DO snapshots weekly (DigitalOcean managed). Snapshots retained for 4 weeks.
- **Issuer seed:** offline backup in 1Password + paper copy in Sami's safe. Retention: same as issuer seed's operational life.
- **Source code:** GitHub (patl4588 org) is authoritative. No off-Git backup — the release tarballs on GitHub Releases + railcall.ai mirror serve as retention for the shipped state at each version.

8. What we do NOT retain

Stated explicitly so absence cannot be misread:

- **Customer workflow inputs or outputs.** The engine runs on the customer's box; results never traverse our infrastructure. This is architectural, not policy — see SRA §1 "Key architectural fact."
- **Customer PHI or PII of end-users.** Blocked at construction by `phi_guard` before it can even reach a signed receipt. See Encryption Policy §5.
- **Raw API keys.** Stored as `api_key_hash` (SHA-256) after minting; the raw key is only visible on the success page immediately post-purchase and cannot be re-revealed.
- **Session tokens.** Process-lifetime only; discarded on studio exit.

9. Review

At each station release + at least annually. Retention windows are re-checked against current regulatory guidance at each annual review; changes require a corresponding update to the Privacy Policy on `railcall.ai/legal/privacy` at the same time.

10. Related documents

- **Privacy Policy** (`railcall.ai/legal/privacy`) — the customer-facing version of this policy's user rights section.
 - **Encryption Policy** §4 — how the data enumerated here is protected while retained.
 - **Incident Response Policy** §7.2, §7.3 — retention of incident-related data.
 - **BAA_DRAFT.md** — retention obligations we take on when acting as a business associate.
-

Vendor & Subprocessor Management Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC9.2 (Vendor Risk) · HIPAA §164.308(b) Business Associate Contracts + §164.502(e)(1)(ii) Subcontractors · GDPR Art. 28 processor contracts **Companion documents:** BAA_DRAFT.md Exhibit C (subprocessors), Encryption Policy §8, Incident Response Policy §7.4–7.7.

1. Purpose

Enumerate every third party that touches RailCall infrastructure or customer-adjacent data, WHY they touch it, what they can see, and what happens if they fail. Keep the list honest and current — an incomplete subprocessor disclosure is itself a breach of §164.502(e)(1)(ii).

2. Scope

Every third party in the RailCall operational path. Ranked by the sensitivity of what they see.

Out of scope: dev-time tooling that never sees a byte of customer or operator secret (test frameworks, editor plugins, etc.).

3. Current vendor inventory

Sorted by criticality. Each row is a live decision — if we can't defend the entry today, we shouldn't be using the vendor today.

3.1 Render — gateway hosting

- **What they touch:** the `railcall-core` FastAPI service (all paid-tier code paths), managed Postgres holding `consumers` + `seat_reservations` + `processed_events`, service logs, `RAILCALL_ISSUER_SEED` and `STRIPE_SECRET_KEY` as env vars.
- **Why them:** operational simplicity for a two-person team — managed Postgres, TLS termination, deploy-on-push. Alternative considered: DigitalOcean App Platform, AWS Fargate + RDS. Render chosen for lower ops overhead at our scale.
- **Their posture:** SOC 2 Type II (published), GDPR/CCPA-ready per their DPA, US + EU regions available. Encryption at rest on managed Postgres, TLS 1.3 in transit.
- **What they see about customers:** email addresses at signup, `sha256(api_key)` on every request, install pubkeys on seat checkin. Never raw `api_keys` after minting, never workflow data, never PHI (see Encryption Policy §5).
- **What they see about us:** every gateway secret (`RAILCALL_ISSUER_SEED` in particular). This is a hard trust boundary — a Render employee with production DB access could theoretically read the seed. Mitigation: none technical; contract + Render's own SOC 2 controls.
- **Replaceability:** medium. Migrating gateway = redeploying FastAPI on another platform + rewiring env vars + swapping Stripe webhook target. Estimated 1–2 days of work; DNS TTL is the customer-facing risk (short-lived pointer to gateway URL).
- **Review cadence:** on every Render pricing/policy change email; at least annually.

3.2 Stripe — payments + subscription lifecycle

- **What they touch:** every \$, every card, every subscription. Sends `checkout.session.completed`, `customer.subscription.deleted`, etc., to our webhook.
- **Why them:** PCI-DSS Level 1 out of the box, the ubiquitous choice for a US-based SaaS. Alternative considered: Paddle (better VAT handling), Lemon Squeezy. Stripe chosen for API depth + Anthropic-tier reliability.
- **Their posture:** SOC 1/2 Type II, PCI-DSS Level 1, GDPR-compliant DPA.
- **What they see about customers:** everything — name, billing address, card number, purchase intent. RailCall NEVER sees the card number — it only sees a `customer_email`, `customer_id`, and payment status.
- **What they see about us:** live secret keys, webhook signing secret, per-price product configuration.

- **Replaceability:** low. Migrating Stripe = re-onboarding every paying customer (new payment method, new subscription). Would be a business-continuity event.
- **Review cadence:** quarterly review of restricted-key inventory (revoke stale ones); annually on the vendor itself.

3.3 Anthropic — LLM for Probo (dev-only tooling)

- **What they touch:** any prompt Probo's internal agents send (probo agent, evidence describer, third-party vetter, tracker mapping). Probo runs on Sami's laptop — a customer does NOT reach Anthropic through RailCall.
- **Why them:** Claude models power the compliance-drafting loop that made these very policies possible. Alternative: OpenAI (same LLM class). Both providers wired in `PROBOD_AGENT_DEFAULT_PROVIDER`; either can be swapped by env var.
- **Their posture:** SOC 2 Type II, HIPAA-eligible with signed BAA (not signed by RailCall for Probo use because no PHI reaches this path).
- **What they see about customers:** nothing. Probo is fed our own SRA, our own control descriptions, our own policy drafts. Customer data does not leave our infrastructure to reach Anthropic.
- **What they see about us:** the internal state of our compliance program (policies in draft, control implementation notes). Sensitive but not customer-sensitive.
- **Replaceability:** high. Swap `PROBOD_AGENT_DEFAULT_PROVIDER=openai` and set `PROBOD_OPENAI_API_KEY`; no data-model migration needed.
- **Review cadence:** annually; sooner if Anthropic changes its retention policy.

3.4 GitHub (patl4588 organisation) — source + release hosting

- **What they touch:** every line of source code, every release tarball (station-v0.16 and predecessors), every install script mirror, this policy folder.
- **Why them:** default in the ecosystem, developer tools + CI + release hosting all in one. Alternative: GitLab, self-hosted Gitea. GitHub chosen for ecosystem inertia.
- **Their posture:** SOC 2 Type II (Microsoft), extensive breach transparency, MFA + SSO available.
- **What they see about customers:** nothing. Source is not customer data.
- **What they see about us:** every RailCall secret we accidentally commit — mitigated by pre-push review + `install.sh`-level leak gate in the release script that refuses tarballs containing moat artifacts. No `.env` files, no seed material, no vault files are ever committed (verified by `.gitignore` + the leak gate).
- **Replaceability:** medium. Git itself is portable; the ecosystem (Actions, PR flow, gh CLI) is where lock-in lives.
- **Review cadence:** annually; sooner on any GitHub security incident affecting orgs we're part of.

3.5 DigitalOcean VPS 157.230.177.45 — railcall.ai website + installer mirror

- **What they touch:** the Next.js site (`railcall-contrib/website-v2` deployed via pm2), the installer + station tarball mirror, static compliance / legal pages.
- **Why them:** cheap VPS with predictable pricing + snapshot backups. Alternative: Vercel/Netlify for the Next.js side. VPS chosen for full control over the installer mirror (matters for the release ceremony's byte-compare property).
- **Their posture:** SOC 2 (published), ISO 27001, weekly snapshots retained 4 weeks.
- **What they see about customers:** signup emails from the marketing site's contact forms (if any). No paying-customer data goes through the VPS.
- **What they see about us:** the SSH key giving Sami operator access, all static files on disk.
- **Replaceability:** high. Static-site deploy anywhere; installer mirror can move to any HTTP host that supports SNI + reasonable TLS.
- **Review cadence:** annually; on any DO security advisory.

3.6 Cloudflare — DNS + edge for railcall.ai

- **What they touch:** DNS resolution for `railcall.ai`; TLS termination in front of the VPS if that's how it's currently wired (verify against DNS records at review time).

- **Why them:** free/cheap tier, robust DNS, DDoS mitigation.
- **Their posture:** SOC 2, ISO 27001, extensive compliance surface. Notable: DNS logs may include client IPs — treat as processor data under GDPR for EU visitors.
- **Replaceability:** trivial for DNS; higher if we rely on any Cloudflare-specific feature (Workers, R2). Currently: DNS + edge cache only, so trivial.
- **Review cadence:** annually.

3.7 1Password — Sami's secret storage

- **What they touch:** every RailCall root secret Sami holds — issuer seed, deploy SSH keys, Stripe standard-key backups, personal credentials.
- **Why them:** Sami's existing password manager; already used for personal + operator secrets.
- **Their posture:** SOC 2 Type II, end-to-end encrypted with a master passphrase Sami never shares.
- **What they see about customers:** nothing. Customer data is not in 1Password.
- **What they see about us:** everything, encrypted such that a 1Password insider cannot read without Sami's master passphrase.
- **Replaceability:** high. Exports to standard formats; another password manager or offline paper storage is a swap.

4. Onboarding a new vendor

Any addition to §3 requires the following BEFORE production traffic reaches them:

1. **Purpose statement.** One sentence: what data class does this vendor touch, and why is that the smallest amount that works?
2. **Compliance posture check.** SOC 2 / ISO 27001 / HIPAA-eligibility as applicable. If they don't have a relevant certification and they touch customer-sensitive data, they don't get added.
3. **DPA / BAA signed** if they process personal data or PHI. For a US vendor: their standard DPA is usually fine; for EU personal data, verify Standard Contractual Clauses are in place.
4. **Update this policy AND `BAA_DRAFT.md` Exhibit C.** Missing a subprocessor from the BAA disclosure is a §164.502(e)(1)(ii) breach — the disclosure itself is the compliance object.
5. **Notify affected customers.** Any customer with an active BAA gets 30-day written notice via `security@railcall.ai` before the new subprocessor goes live for their data (per BAA §2.5 in the current draft).

5. Offboarding a vendor

1. **Cut off access first.** Rotate credentials, remove SSH keys, revoke API tokens.
2. **Extract our data.** Downloads, exports, or DB dumps from every vendor-hosted store.
3. **Confirm deletion at the vendor.** Every SOC 2 vendor supports "delete my data" — hold them to it in writing.
4. **Remove from this policy AND from `BAA_DRAFT.md` Exhibit C.**
5. **Retain a decision record.** Why we left, when, what data was migrated where.

6. Vendor incidents

Cross-reference `Incident Response Policy` §7.4–7.7 for the per-vendor runbook. Notification obligation applies both ways: - **They notify us:** track it via their status page + any direct email; treat as our own incident from the moment we're aware. - **We notify them:** if a RailCall incident affects data they process (e.g., our webhook secret leaks), tell Stripe / Render / whoever is affected. Ask them to confirm no downstream impact.

7. Review cadence

- **Per-vendor:** as listed in §3 (mostly annual).

- **Whole policy:** annually + on any material change to a vendor (pricing tier change, ownership change, compliance certification lapse).
 - **Customer-facing subprocessor list** (railcall.ai/legal/subprocessors — TO BE CREATED): updated within 30 days of any §3 change, per common DPA obligations.
-

8. Related documents

- **BAA_DRAFT.md** Exhibit C — the customer-facing legal disclosure of these subprocessors. This policy is the operator-facing detail; Exhibit C is the customer-facing summary.
 - **Encryption Policy** §8 — what encryption we entrust to each of them.
 - **Incident Response Policy** §7.4–7.7 — how we respond when each of them fails.
 - **Access Control Policy** §6 — who on our side has access to each vendor's console.
-

Business Continuity & Disaster Recovery Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC7.5 (Recover from disruptions) · CC9.1 (Risk mitigation for business disruptions) · HIPAA §164.308(a)(7) Contingency plan · ISO 27001 A.5.30 (ICT readiness for business continuity) **Companion documents:** Incident Response Policy §7 · Encryption Policy §7 · Data Retention & Deletion Policy §7.

1. Purpose

State what has to keep working when things break, how long we're willing to be down, and the specific playbook to get back. Every RTO/RPO here is one this team can actually meet with the operational reality we have — not aspiration, not marketing.

2. Business-critical services

Ranked by what actually stops the business if it's down.

#	Service	Stops what if down	Owner
1	Local station on customer machines	Every customer's product	Customer (we can't affect this once shipped; recovery = reinstall)
2	<code>railcall.ai/install.sh</code> + <code>railcall_station.tar.gz</code> mirror	New signups + reinstalls	VPS <code>157.230.177.45</code> + <code>git pull</code> on the box
3	Gateway <code>railcall-core.onrender.com</code>	Paid tier: mint/activate/checkin/webhook	Render
4	Gateway Postgres	All paid-tier state — subscriptions, seat reservations, meter	Render managed
5	Stripe	New checkouts, subscription lifecycle events	Stripe
6	<code>railcall.ai</code> marketing site	Sign-up conversion, docs pages	Same VPS as #2
7	GitHub Releases + repo	Cutting new station releases, install.sh distribution	GitHub
8	Anthropic API (Probo only)	Our own compliance drafting velocity	Anthropic (dev-only, no customer impact)

Deliberately absent: any "internal SaaS" (there is none), any "employee laptop" (Sami's box has the issuer seed backup but is not itself business-critical because the seed is also in 1Password + paper).

3. Recovery objectives

RTO = how long a service can be down before it's a problem. RPO = how much data loss is acceptable at recovery.

Service	RTO	RPO	Justification
Local station (per customer)	15 min	0 (data lives on customer's box, not affected by our outages)	Customer runs the installer; nothing on our side to restore
<code>install.sh</code> + tarball mirror	4 h	0 (installer/tarball are static, versioned; recover from GitHub Release + git pull)	Blocks new signups; existing customers unaffected
Gateway (paid-tier code path)	4 h	0 (stateless code; state is Postgres)	Paid customer already activated is unaffected; new mints blocked
Gateway Postgres	24 h	24 h (Render managed backup cadence)	Losing 24 h of <code>consumers</code> rows = losing recent signups (recoverable from Stripe webhook replay); losing <code>seat_reservations</code> is self-healing on next checkin
Stripe (their outage)	Their SLA	Their SLA	Nothing we can restore; documented for transparency
Marketing site	24 h	0 (git-versioned)	Not paid-tier blocking; conversion loss only
GitHub	Their SLA	Their SLA	Our critical dependency; a full GitHub outage delays station releases until they recover or we mirror elsewhere (\$5.6)
Anthropic (Probo)	N/A	N/A	Dev tooling; no customer impact

4. Backups + snapshots — what we can restore from

Data	Where the backup lives	Backup cadence	Restore verified?
Gateway Postgres	Render managed backup surface	Render default	Not verified end-to-end by us; relies on Render restore procedure. Action item: run one real restore drill against a staging DB before EOY 2026.
VPS <code>157.230.177.45</code>	DigitalOcean snapshots	Weekly, 4-week retention	Not verified. Action item: annual snapshot-restore drill.
Issuer seed	1Password + offline paper in Sami's safe	Manual, on rotation	Verified — the mint tool's <code>check</code> subcommand recomputes the pubkey from the stored seed so a restored seed can be validated without deploying.
Source code	GitHub (patl4588 org)	Every push	Verified continuously — every fresh clone IS a restore.
Signed release tarballs	GitHub Releases + railcall.ai mirror	On every station cut	Verified — the release ceremony byte-compares the downloaded tarball against the local build before pinning <code>install.sh</code> .
This compliance folder	Git (railcall-core <code>compliance/</code>) + Probo	Every commit / every policy edit	Verified — Git IS the retention layer.

5. Disaster runbooks

Each runbook is scoped by "what failed", not by "what caused it." The cause matters for the post-mortem, not for the restore.

5.1 Issuer signing seed lost (no compromise suspected)

Cross-reference: Encryption Policy §7 + Incident Response Policy §7.1.

RTO: 8 hours (station release cycle is the bottleneck). **RPO:** 0 for existing entitlements (they keep verifying against the pinned key until expiry). New mints blocked until the new key is in the pin.

Steps: 1. Mint new keypair: `python3 tools/mint_issuer_keypair.py mint --seed-out ~/railcall-issuer-seed-YYYYMMDD.txt`. Back up to 1Password + paper BEFORE proceeding. 2. Edit `ISSUER_PUBKEY_HEX` in `railcall-engine/workbench/primitives/entitlement.py`. Commit + push. 3. Cut new station release (`station-vN+1`): overlay engine into

~/railcall/station/ , run `bash scripts/build_station_tar.sh` , verify leak gate + fresh-install smoke. 4. `gh release create station-vN+1` with the tarball. Byte-compare downloaded asset against the local build. 5. Update `STATION_SHA` + `STATION_URL` in `railcall-core/install.sh` and `install.ps1` . Push. 6. Mirror to `railcall-contrib/website-v2/public/` (both installer + tarball). Push. 7. `ssh sami@157.230.177.45 && cd ~/railcall-contrib && git pull && pm2 restart railcall-website` . 8. Verify `curl https://railcall.ai/install.sh` returns new sha, and public tarball sha matches. 9. Set `RAILCALL_ISSUER_SEED` on Render service (Encryption Policy §4.2 — direct service-level, not env group). 10. Verify `curl https://railcall-core.onrender.com/v1/issuer/pubkey` returns the new pubkey. 11. Email every paying customer: "reinstall via `curl -fsSL https://railcall.ai/install.sh | bash` , then `railcall activate` to re-mint your entitlement."

5.2 Render gateway complete outage

RTO: 4 hours (dependent on Render's own recovery). **RPO:** 0 (state is durable in managed Postgres).

Steps: 1. Confirm scope via <https://status.render.com>. 2. If our service specifically is down (not Render broadly): check Render dashboard for failed deploys, roll back to the last known-good deploy. 3. If Render broadly is down: post to Discord + Twitter. Nothing else we can do — this is Render's runbook. 4. Free-tier installs are unaffected (they don't depend on the gateway). Communicate this in the outage notice so free users don't panic. 5. Paid customers with active entitlements are unaffected until expiry. If an entitlement expires during the outage, extend it manually via email after service returns. 6. Post-outage: replay any Stripe webhooks that failed during the window (Stripe retries automatically for 72 h).

5.3 Gateway Postgres data loss

RTO: 24 hours. **RPO:** 24 hours (Render managed backup cadence).

Steps: 1. Immediately halt new writes (put gateway in maintenance mode via a feature flag or manual deploy of a maintenance-page image). 2. Restore latest Render backup to a NEW DB instance. 3. Point gateway env `DATABASE_URL` at the restored instance. Redeploy. 4. Replay Stripe webhooks for the period between backup point and outage. `checkout.session.completed` events are idempotent on `cs:<session_id>` so replay is safe. 5. Seat reservations are self-healing — every install pings every 6 h, table reconstructs within a TTL cycle. 6. Send affected customers (identified by absence of expected `consumers` row) a mint-retry link so they get their entitlement re-issued.

5.4 VPS 157.230.177.45 unreachable or lost

RTO: 4 hours (DNS TTL is the bottleneck). **RPO:** 0 for installer/tarball (they live in Git + GitHub Releases).

Steps: 1. Confirm scope via DO console. If VPS unrecoverable: provision a new droplet from the latest DO snapshot. 2. If snapshot restore is fine: point `railcall.ai` A record at the new IP. DNS propagates in ≤ 5 min if TTL is low. 3. If snapshot unavailable: fresh droplet, install Node + pm2, `git clone git@github.com:patl4588/railcall-contrib.git` , checkout `feat/website-v2` , `npm ci && npm run build` , `pm2 start ...` . ~1 hour. 4. Add Sami's SSH key + Pat's root key. Verify installer + tarball are byte-identical to the GitHub Release copy before flipping DNS. 5. Post-recovery: send a "we moved boxes" note only if anyone noticed.

5.5 Stripe outage

RTO: Stripe's SLA. **RPO:** Stripe's SLA.

Nothing we can do besides communicate. Existing subscribers keep being charged by Stripe independently. New checkouts blocked until they're back. Add a banner to the pricing page pointing to Discord for updates.

5.6 GitHub outage (or GitHub compromise affecting our org)

RTO: their SLA for outages; 8 h to switch to mirror on compromise.

Steps for outage: 1. Wait for GitHub to recover. Nothing we can do; every dev depends on GitHub for something.

Steps for org compromise (unauthorized push to `patl4588/railcall-core` or `*-engine`): 1. Rotate every push token. Enforce SSH-key-only + MFA everywhere. 2. Force-push last known-good commit from any local checkout that predates the compromise. 3. Cut a new station release ONLY after auditing every commit since the compromise window. 4. Notify anyone who installed from a compromised commit window — if the tarball on the release DIFFERS from what our local build produces, treat this as a supply-chain event per Incident Response Policy §7.7.

5.7 Sami unavailable (primary operator down)

Sober edge case for a two-person team. Steps to make it survivable:

1. Every root secret Sami holds must also be in a place Pat can reach IF the situation warrants — 1Password shared vault, or sealed envelope in a physical location Pat knows about. Currently: 1Password backup of issuer seed exists; sealed physical backup is an **action item**.
2. Pat has SSH access to the VPS with a root key. Verify annually that his key still works.
3. GitHub org owner is Pat; Sami is admin. If Sami is unavailable, Pat can still merge/tag/release.
4. Render is admin-shared. Stripe is admin-shared (as of 2026-07-21).
5. Anthropic account is Sami's — a Pat-usable API key is an **action item**.

Documented explicitly: if Sami is unavailable for > 24 h and no incident is in progress, business continues without material impact. If Sami is unavailable DURING an active SEV-1, Pat + a Signal call to Claude Code (which has the runbooks) is the fallback.

5.8 Sami's laptop lost / stolen

RTO: 4 hours (mostly re-provisioning). **RPO:** 0 (nothing customer-critical lives ONLY on Sami's laptop).

Steps: 1. From another box, revoke Sami's SSH keys everywhere (GitHub, VPS, Render). 2. Rotate 1Password master passphrase (whole vault re-encrypted). 3. Rotate every API key Sami had — Stripe (all of them), Anthropic, GitHub PATs, Render CLI token if any. 4. Verify the issuer seed is still safe (1Password is E2E encrypted so a laptop grab doesn't reveal it; paper backup in the safe is unaffected). 5. Provision a new box. Sami reinstalls the dev tooling from public sources — no proprietary tooling.

6. Testing

- **Tabletop exercises:** one per quarter, alternating scenarios from §5. Log as SEV-4 (tabletop) incidents per Incident Response Policy §10.
- **Live drill:** annual issuer-seed rotation (§5.1) — the natural exercise because it's the highest-blast-radius runbook AND it's periodically valuable to actually do (freshness of the ceremony).
- **Backup restore drill:** annual Postgres restore into a staging DB (§4 action item).

7. Business impact analysis (BIA) — summary

Financial exposure of each scenario, roughly:

Scenario	Financial impact per day of outage	Reputational impact
§5.1 issuer seed lost	Zero direct revenue impact (existing subs continue); reputation risk if we handle the notification badly	Medium — every paying customer has to act
§5.2 gateway outage	~\$daily-signups-cost — depends on signup rate	Low — free tier unaffected
§5.3 Postgres data loss	~1 day of signups lost + customer trust hit	Medium — customers noticing missing account state
§5.4 VPS lost	~\$signups-during-outage + install-time friction	Low — mostly transparent to existing users
§5.5 Stripe outage	Zero direct (they hold their own state); new signups blocked	Low — attributable to Stripe
§5.6 GitHub compromise	Existential if pinned code is tampered undetected	Very high
§5.7 Sami unavailable	Slowed velocity; no direct outage	Depends on duration
§5.8 laptop lost	Zero if credentials rotate cleanly; existential if issuer seed also compromised in transit	Medium

8. Review

Whole policy reviewed at each station release + at least annually. Action items in §4 (backup restore drill) and §5.7 (Pat operational continuity) are tracked in Probo and re-verified quarterly.

9. Related documents

- Incident Response Policy §7 — how we detect + triage what §5 recovers from.
 - Encryption Policy §7 — the key-lifecycle piece of §5.1.
 - Data Retention & Deletion Policy §7 — what backups exist and their retention.
 - Access Control Policy §6 — who has the credentials each recovery step requires.
 - Vendor Management Policy §3 — the per-vendor posture that determines each RTO/RPO.
-

Change Management Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC8.1 (Change management) · CC7.1 (Design + implementation of controls) · ISO 27001 A.8.32 · HIPAA §164.312(a)(1) reasonable safeguards **Companion documents:** Access Control Policy · Incident Response Policy · BC/DR Policy · Encryption Policy §7.

1. Purpose

Describe how changes actually get from a developer's box to production without breaking customer trust. This is not aspirational — it's the pipeline we actually run today, written down so a reviewer can trace any deployed byte back to a specific commit + specific test result + specific human decision.

2. Scope

- **Code:** every commit to `railcall-core`, `railcall-engine`, `railcall-contrib`.
- **Station releases:** every station tarball published on GitHub Releases + mirrored to railcall.ai.
- **Gateway deploys:** every Render auto-deploy triggered by push to `railcall-core:main`.
- **Website deploys:** every pm2 restart on VPS `157.230.177.45`.
- **Configuration changes:** env-var edits on Render, DNS record edits on Cloudflare, Stripe Price/Product edits, GitHub org / repo settings.

OUT of scope: local dev-environment tweaks that never leave a developer's box.

3. Change classes

Class	Definition	Example	Approval	Testing gate
Standard	Additive, no state migration, no protocol change	Adding a new CLI command, docs edits, new test	Author self-review	Existing test suite must pass
Enhancement	Behavior change, backwards-compatible	Adding <code>seat_count</code> column with NULL default; new endpoint that returns 401 for anonymous	Author + one reviewer (Sami if Claude authored)	Existing + new dedicated tests
Breaking	Not backwards-compatible; requires customer action	Rotating <code>ISSUER_PUBKEY_HEX</code> ; changing entitlement schema; removing an endpoint	Sami explicit sign-off + release ceremony	Full regression + customer-facing communication plan
Emergency	Fix in production for an active incident	Rolling back a bad deploy; hotfix for exploited vuln	IC decision, retroactive review within 24 h	Minimum viable test at deploy time; full test in follow-up PR

4. The pipeline — what actually runs

For a Standard or Enhancement class change to `railcall-core`:

1. **Code + tests locally.** Author writes the change and at least one dedicated test that would have failed pre-change and passes post-change.
2. **Local test suite.** `python3 -m pytest tests/ -q` + any suite specific to the file touched (e.g., `test_seat_checkin.py` for seat code, `test_paid_full_e2e.py` for full-chain changes).

3. **Commit with an evidence-forward message.** Message describes WHY (the invariant being defended or the gap being closed), not just WHAT.
4. **Push to `railcall-core:main`.** Triggers Render auto-deploy.
5. **Watch the deploy.** Render dashboard shows deploy status. Deploy failures roll back automatically.
6. **Post-deploy smoke.** `curl` the affected endpoint(s) with a known-shape request; verify expected HTTP code + response shape. Log the smoke result to the incident note if the change is Enhancement or above.

For `railcall-engine`:

1. Same 1–3 above.
2. Push to the working branch (`feat/paid-tier-entitlement` currently). Engine does NOT auto-deploy; changes reach users only via a station release (§5).

For `railcall-contrib` (website + installer mirror):

1. Same 1–3.
2. Push to `feat/website-v2`.
3. `ssh sami@157.230.177.45 && cd ~/railcall-contrib && git pull`.
4. If a `.tsx` / `.ts` changed: `cd website-v2 && npm run build`.
5. If a file under `website-v2/public/` changed: `pm2 restart railcall-website` (Next.js caches the public/ listing at boot; changed files need a restart to serve).
6. Verify the affected URL via `curl` from outside the VPS.

5. Station release ceremony (Breaking-class only)

Station releases are the highest-blast-radius change class we make. Every user's install eventually runs the code we cut here.

Full ceremony (mirrors what was executed for `station-v0.16` on 2026-07-21):

1. **Verify engine is on the right branch + commit** (`feat/paid-tier-entitlement` currently, but check per-release).
2. **Overlay engine into station tree:** `rsync -a --exclude=__pycache__ --exclude='*.pyc' --exclude='tests/' --exclude='.railcall_workspace/' "$ENGINE/workbench/" "$STATION/workbench/"`.
3. **Verify pin (if rotating):** `grep '^ISSUER_PUBKEY_HEX' "$STATION/workbench/primitives/entitlement.py"` matches the intended value.
4. **Build tarball:** `STATION_SRC=... OUT=... RELEASE_TAG=station-vN ENGINE_COMMIT=... CORE_COMMIT=... bash scripts/build_station_tar.sh`.
5. **Verify leak gate passed** — the build script refuses to publish if any factory/moat artifact is included. Non-optional.
6. **Verify tarball has no secrets:** `tar -tzf <tarball>` and `grep` for `keys.local.json`, `.env`, `token.json` — must return zero matches.
7. **Boot the extracted station on an unused port** (`STUDIO_PORT=8815 python3 studio_server.py`), hit `/api/version` — must return the new `release_tag`.
8. **sha256 the tarball.** `shasum -a 256 "$OUT"`.
9. **Publish to GitHub Releases:** `gh release create station-vN "$OUT" --title "..." --notes "..."`.
10. **Rename the asset to `railcall_station.tar.gz`** (unversioned, per `install.sh` URL convention).
11. **Byte-compare downloaded release asset against local build.** Must match. If not: DO NOT PIN.
12. **Update `STATION_SHA` + `STATION_URL` in `install.sh` AND `install.ps1`.**
13. **Commit + push `railcall-core`.**
14. **Mirror to contrib:** `cp` installer + tarball into `railcall-contrib/website-v2/public/`, commit, push `feat/website-v2`.
15. **Deploy website:** `ssh sami@... && cd ~/railcall-contrib && git pull && pm2 restart railcall-website`.
16. **Verify public endpoints:** `curl https://railcall.ai/install.sh | grep STATION_SHA` matches the new sha; `shasum -a 256 <(curl https://railcall.ai/railcall_station.tar.gz)` matches.

17. **Fresh-install smoke:** in an isolated `HOME`, `curl -fsSL https://railcall.ai/install.sh | HOME=... bash`. Run `railcall version` — must show the new `station-vN ✓ latest`.

This ceremony is documented step-for-step because it's the one an operator under stress has to follow exactly. Any deviation → don't ship.

6. Approval + separation of duties

Small team reality: Sami is often the author AND the reviewer AND the deployer. That's not a separation-of-duties failure if it's honestly documented. What IS required:

- **Machine-enforced gates precede human-enforced ones.** Tests + leak gate + sha byte-compare are the actual safety net; the human review confirms them, doesn't substitute.
- **Standing merge authorization** (per `feedback_merge_authorization.md`): merges on green CI across repos Sami controls proceed without asking each time. This is a documented, deliberate policy for a two-person team.
- **Breaking changes still require Sami's explicit sign-off** even under standing auth. See §3.
- **Emergency changes require post-hoc review** within 24 h. Log in the incident note; document in the next weekly sync.

When headcount grows past 2, this section should tighten — separate authors from approvers on Breaking-class + revisit standing auth.

7. Rollback

Every deploy has a rollback path. Documented per surface:

- **Render gateway:** Render dashboard → "Manual Deploy" → pick the previous known-good deploy. ~2 min.
- **Station release:** revert the `install.sh` pin change (commit + push); users' next install picks up the previous tarball. Users who already installed the bad version need a reinstall (`curl -fsSL https://railcall.ai/install.sh | bash`).
- **VPS website:** `ssh sami@... && cd ~/railcall-contrib && git reset --hard <prev-commit> && npm run build && pm2 restart railcall-website`.
- **Engine commits:** git revert on the working branch. Only reaches production via a subsequent station release, so effectively delayed until §5 is repeated.
- **Stripe / Render env-var change:** re-edit the variable to the previous value; save; wait for redeploy.

Rollback is always preferable to "roll forward through the fire" during an active incident. Take rollback first, root-cause second.

8. Emergency changes — exceptions to §4

An Emergency change bypasses parts of §4 to restore service. Constraints:

- **Only the IC can invoke emergency mode.** Log the invocation in the incident note.
- **Skip is documented** — every skipped step (tests, review, ceremony sequence) is listed in the incident note with the reason.
- **Follow-up PR within 24 h** brings the code back into full compliance (tests written, formal review, any missed docs updated).
- **Post-mortem** covers the emergency change specifically: was the shortcut justified, could it have been avoided.

Examples of legitimate emergency: rolling back an `install.sh` pin that broke every fresh install. Not emergency: skipping tests because the fix is "obvious."

9. Post-change verification

For every Enhancement + Breaking change, post-deploy verification includes:

1. **Endpoint smoke:** hit affected URLs, verify expected responses.

2. **Log check:** watch Render logs (or `pm2 logs`) for the first 5 min post-deploy. Any new error class → investigate.

3. **Update any relevant Probo controls or measures** — a change that closes a gap should flip that control's status from `NONE` to `INITIAL` / `MANAGED` / `DEFINED` (see Risk Management Policy for the state machine).

Every deployment leaves an audit trail: the git commit is the change record; the Render deploy is the deploy record; the smoke verification is captured in the commit message (for Standard) or an incident note (for Emergency).

10. Configuration change tracking

Not everything is in Git. What's not:

Change	Where it lives	How we track it
Render env vars (all <code>PROBOD_*</code> , <code>RAILCALL_*</code> , <code>STRIPE_*</code> , <code>DOMAIN_URL</code>)	Render dashboard	Documented in <code>render.yaml</code> (variable names + <code>sync:false</code> marker); values held in 1Password + Render itself
DNS records	Cloudflare	Cloudflare's own change log
Stripe Products + Prices	Stripe dashboard	Stripe's own audit log; Price IDs referenced in code
GitHub org settings	GitHub	GitHub audit log

Rule: if a value in one of these systems changes and code doesn't, the change is still a Change under this policy — document in the incident note or a "config-change" issue in the repo.

11. Review

Whole policy reviewed at each station release + at least annually. §5 (station release ceremony) is the section most likely to drift — if we automate any step, this document is the source that must be updated first, not last.

12. Related documents

- Incident Response Policy §8 — the emergency-change escalation path.
- BC/DR Policy §5.1 + §5.4 — the recovery paths that station release + VPS deploy runbooks share with change management.
- Access Control Policy §6 — who has the credentials each deploy step requires.
- Vendor Management Policy §3.1, §3.5 — Render + VPS specifics.

Risk Management Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC3.1–3.4 (Risk assessment) · HIPAA §164.308(a)(1)(ii)(A/B) (Risk analysis + risk management) · ISO 27001 clause 6.1 · NIST SP 800–30r1 (Risk assessment) **Companion documents:** HIPAA SRA v1 (compliance/HIPAA_SRA_v1_2026-07-21.md) · Incident Response Policy §3 (severity classes) · every other policy in Probo (each closes a specific risk).

1. Purpose

Turn the point-in-time SRA into an operating risk register that gets used, not filed. Every risk here has a named owner, a treatment strategy, and a review date — so decisions about tradeoffs are made deliberately, not accidentally by omission.

2. Scope

- Every risk identified in the SRA (§3.1–3.8 of `HIPAA_SRA_v1_2026-07-21.md`).
- Every gap listed as `GAP` or `PARTIAL` in SRA §4.
- Every accepted operational limit documented in this policy set (Encryption §5, BC/DR §5.5, Vendor Mgmt §3.1).
- New risks discovered during incidents (per Incident Response Policy §9 post-mortems).

3. Risk-scoring model

Two numbers per risk. Both on a 1–5 scale, both deliberately coarse — a two-person team can't defend fine gradations honestly.

Likelihood — how probable is realization within the next 12 months: - 1 Rare (requires unusual conditions we don't foresee) - 2 Unlikely (possible but no known driver) - 3 Possible (a known driver exists but no active pressure) - 4 Likely (active pressure or known-vulnerable path) - 5 Almost certain (already happened, or actively being tried)

Impact — if realized, blast radius: - 1 Negligible (< 1 h of ops noise) - 2 Minor (one customer affected, contained) - 3 Moderate (multiple customers or one enterprise, no data loss) - 4 Major (data loss, regulatory notification, revenue hit) - 5 Severe (existential — brand-eroding breach, trust root compromise)

Score = Likelihood × Impact. 1–25.

Bands: - 15–25 = **HIGH** — treat within 30 days OR document explicit acceptance with sign-off - 8–14 = **MEDIUM** — treat within 90 days OR accept with review at next quarter - 1–7 = **LOW** — track, review annually

4. Treatment strategies

Named per NIST/ISO conventions. Every risk in §6 chooses one:

- **MITIGATE** — reduce likelihood and/or impact via a control. Requires an owner + a target date. This is what most P0 remediations did.
- **ACCEPT** — acknowledge and continue as-is because mitigation cost exceeds expected loss OR mitigation would break another property (e.g., unattended operation). Requires an owner + a review date.
- **AVOID** — remove the source of the risk. Example: not accepting PHI on the gateway (§164.312 transmission gap was AVOIDED by never accepting bodies).
- **TRANSFER** — shift the risk to a third party via contract or insurance. Example: relying on Stripe's PCI-DSS Level 1 posture for card handling.

5. Ownership

Every risk has ONE owner. Currently every entry defaults to Sami (Security Officer). When headcount grows, ownership should be distributed — the Owner column here is the enforcement mechanism for that transition.

6. Risk register — v1 baseline

Structured so it can be imported into Probo's risk module as `addRisk` entries. Same numbering as SRA §3 so cross-reference is trivial.

R-01 — Local file-system exposure of signing seed (SRA §3.1)

- **Likelihood:** 2 (backup/sync exposure was a real driver pre-P0-1)
- **Impact:** 5 (receipt trust root compromised → every receipt from that install becomes suspect)
- **Score:** 10 MEDIUM
- **Treatment:** MITIGATED (P0-1 closed 2026-07-20 for station, 2026-07-21 for CLI)
- **Residual:** same-user malware can still reach keychain. LOW residual score (Likelihood 2 × Impact 3 = 6). ACCEPTED with review at annual SRA refresh.
- **Owner:** Sami
- **Evidence:** Probo measure `P0-1`; Encryption Policy §4.1; `test_seed_store.py` 16/16.

R-02 — Audit log tamper (SRA §3.2)

- **Likelihood:** 2
- **Impact:** 4 (§164.308(a)(1)(ii)(D) audit requirement met in name but not substance)
- **Score:** 8 MEDIUM
- **Treatment:** MITIGATED (P0-2 closed via `audit_chain`)
- **Residual:** tail-truncation by install-key holder undetectable. Impact 4 unchanged (still §164.308 gap for that case) but Likelihood drops to 1 (requires local root access already). Score 4 LOW. ACCEPTED pending off-box witness on the roadmap.
- **Owner:** Sami
- **Evidence:** Probo measure `P0-2`; `audit_chain.py` + `test_audit_chain.py` 17/17.

R-03 — PHI egress via generic-named fields (SRA §3.3)

- **Likelihood:** 4 (customers with clinical data + generic field names is exactly the shape a healthcare deployment takes)
- **Impact:** 5 (SSN survived redaction pre-P0-3 — worst-case egress path)
- **Score:** 20 HIGH → post-remediation revisit
- **Treatment:** MITIGATED (P0-3 closed via `phi_guard` + auto-escalation + install-level PHI mode)
- **Residual:** personal names in free text still not detectable; escalation is opt-in per install. Likelihood drops to 2 (only realized if PHI mode is NOT enabled AND field names are all generic AND names appear in free text). Impact still 4 (single-record). Score 8 MEDIUM. ACCEPTED with mandatory operator notice at BAA-signature time (installer must set `RAILCALL_PHI_MODE=on` before touching PHI).
- **Owner:** Sami
- **Evidence:** Probo measure `P0-3`; `phi_guard.py` + `test_phi_guard.py` 29/29.

R-04 — Plaintext PII in audit records (SRA §3.4)

- **Likelihood:** 4 (already realized in 2 of 49 real records)
- **Impact:** 2 (email is identifying but not clinical)
- **Score:** 8 MEDIUM
- **Treatment:** MITIGATED (P0-4 closed same commit as P0-2)

- **Residual:** any new "sensitive-shaped" field name added to the code path could regress. Score 2 LOW.
- **Owner:** Sami
- **Evidence:** Probo measure `P0-4`.

R-05 — Cross-install entitlement replay (SRA §3.7)

- **Likelihood:** 3 (natural human behavior — "I paid, why can't I use it on my second machine?")
- **Impact:** 3 (revenue leak proportional to how much replay happens)
- **Score:** 9 MEDIUM
- **Treatment:** MITIGATED
- **Residual:** LOW (Likelihood 1 × Impact 3 = 3). Entitlements are bound to `install_pubkey`; seat cap enforced server-side; TTL-based prune keeps the accounting live.
- **Owner:** Sami
- **Evidence:** entitlement.py `verify_entitlement`; `/v1/seat/checkin` in cloud_gateway.py; test_activate_e2e.py step 3a; test_paid_full_e2e.py E1-E4.

R-06 — Studio approval token compromise (SRA §3.8)

- **Likelihood:** 2 (requires a compromised browser process on the customer's box)
- **Impact:** 4 (a self-approved external send bypasses dual control)
- **Score:** 8 MEDIUM
- **Treatment:** MITIGATED (approve token printed to terminal only, never templated into any served page)
- **Residual:** LOW (Likelihood 1 × Impact 4 = 4). ACCEPTED — an already-compromised browser has bigger problems than an unlocked studio.
- **Owner:** Sami

R-07 — Paid-tier billing integrity on cancel (SRA §3.6)

- **Likelihood:** 3 (customers cancel — it's normal)
- **Impact:** 2 (cancelled customer keeps consuming seats they don't pay for)
- **Score:** 6 LOW pre-treatment
- **Treatment:** MITIGATED (Stripe `customer.subscription.deleted` + `.updated` webhook branch flips `status=inactive`, `seat_count=0`)
- **Residual:** MINIMAL. Score 2 LOW.
- **Owner:** Sami
- **Evidence:** cloud_gateway.py lifecycle handler; test_stripe_lifecycle.py 11/11.

R-08 — Contract-violation PHI on `/v1/attestation/countersign` (SRA §5.3)

- **Likelihood:** 2 (requires deliberate protocol violation by caller)
- **Impact:** 3
- **Score:** 6 LOW
- **Treatment:** MITIGATED by explicit param typing (endpoint has no body field). ACCEPTED — a future edit could regress this; process control is code review.
- **Owner:** Sami

R-09 — Same-user malware defeats keychain (SRA §5.1 / Encryption §7)

- **Likelihood:** 2 (customer's box malware is not our attack surface, but it's a real environment risk)
- **Impact:** 5 (signing seed compromise per-install)

- **Score:** 10 MEDIUM
- **Treatment:** ACCEPTED — mitigation (hardware token / per-signature passphrase) breaks unattended operation, which is the point of a local automation engine.
- **Owner:** Sami (documented + surfaced via `railcall doctor`)
- **Review:** annually — if hardware tokens become ubiquitous enough to enable optionally, re-open.

R-10 — Audit-chain tail truncation (SRA §5.2)

- **Likelihood:** 2 (requires install-key holder acting against their own compliance interest)
- **Impact:** 3 (post-incident evidence weakened)
- **Score:** 6 LOW
- **Treatment:** ACCEPTED pending off-box witness. Test 4a asserts the limit publicly so no auditor is misled.
- **Owner:** Sami
- **Target:** Off-box witness on roadmap; not-committed date.

R-11 — Issuer seed loss (BC/DR §5.1)

- **Likelihood:** 1 (backup exists in 1Password + paper)
- **Impact:** 5 (existential — every outstanding entitlement becomes unreplaceable)
- **Score:** 5 LOW
- **Treatment:** MITIGATED via backup redundancy; runbook proven in BC/DR §5.1.
- **Owner:** Sami

R-12 — Third-party outage (BC/DR §5.2, §5.5, §5.6)

- **Likelihood:** 4 (SaaS outages happen)
- **Impact:** 2 (bounded by vendor SLA + our RTO)
- **Score:** 8 MEDIUM
- **Treatment:** ACCEPTED. TRANSFERRED to vendor SLAs (Render, Stripe, GitHub). Free-tier installs unaffected by design.
- **Owner:** Sami
- **Review:** on any vendor SLA change.

R-13 — Sami unavailable (BC/DR §5.7)

- **Likelihood:** 3 (happens — vacation, illness, travel)
- **Impact:** 3 (velocity slows; SEV-1 response degraded)
- **Score:** 9 MEDIUM
- **Treatment:** PARTIALLY MITIGATED — Pat has SSH + Render + Stripe admin. Sealed physical backup of issuer seed is an OPEN action item; Pat-usable Anthropic API key is an OPEN action item.
- **Owner:** Sami
- **Target date for open items:** end Q3 2026.

R-14 — Sami laptop lost/stolen (BC/DR §5.8)

- **Likelihood:** 2
- **Impact:** 4 (short window of exposure until rotation)
- **Score:** 8 MEDIUM
- **Treatment:** MITIGATED via runbook (§5.8). 1Password E2E encryption is the primary control; issuer seed paper backup is the ultimate fallback.
- **Owner:** Sami

R-15 — Overclaim on compliance page (regulatory + reputational)

- **Likelihood:** 5 (already realized pre-2026-07-21 — /enterprise carried "HIPAA BAA available" and "SOC 2 in progress" without evidence)
- **Impact:** 4 (misleading advertising is enforceable by state AGs + destroys enterprise-buyer trust when caught)
- **Score:** 20 HIGH → post-remediation revisit
- **Treatment:** MITIGATED (softened /enterprise language in 9696012 ; SRA + policies now provide the evidence base honestly).
- **Residual:** LOW (Likelihood 1 × Impact 4 = 4) — process control is the "no fake-green" rule (see feedback_railcall_culture.md).
- **Owner:** Sami

R-16 — Multi-user host deployment without §164.312(d) human auth

- **Likelihood:** 2 (specifically healthcare — the customer class most likely to run on shared hosts)
- **Impact:** 3
- **Score:** 6 LOW
- **Treatment:** ACCEPTED — customer's own OS user boundary is the compensating control. Multi-user hardening on product roadmap; when it lands, R-16 revisits.
- **Owner:** Sami

7. Register maintenance

- **Adding a risk:** on discovery (incident, code review, external report). Fill all fields; missing owner = policy violation.
- **Updating a score:** when new evidence changes Likelihood or Impact. Log the reason in the register history.
- **Closing a risk:** when treatment lands + verification confirms residual ≤ LOW. Move to §8 archive with the closing evidence.
- **Reviewing accepted risks:** at least annually, or on any change to the accepting context.

8. Quarterly review

Every quarter, this policy owner walks §6 and:

1. Confirms every MITIGATED entry's evidence still holds (test suites still green; files still exist; commits still reachable).
2. Confirms every ACCEPTED entry's accepting rationale still holds.
3. Confirms every OPEN action item has moved (or is escalated).
4. Adds any risks discovered in incidents that quarter.
5. Removes any risks made obsolete by architectural change (documented, not silently deleted).

Log the review as an incident-note-style artifact in ~/reviews/YYYY-QN-risk-review.md . Reviewer signs at the bottom.

9. Related documents

- **HIPAA SRA v1** — the source material for §6 rows R-01 through R-08.
 - **Incident Response Policy §9** — post-mortems that spawn new risks feed into §7.
 - **BC/DR Policy §7** — the business impact analysis that calibrates Impact scores.
 - **Encryption + Access Control + Vendor Mgmt + Change Mgmt Policies** — each closes a specific R-nn or documents an ACCEPTED one.
-

Access Review Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.3 (Manages logical access modifications) · HIPAA §164.308(a)(4)(ii)(C) periodic review · ISO 27001 A.5.18 (Access rights review) **Companion documents:** Access Control Policy (§6 — the systems inventory this reviews) · Vendor Management Policy (§3 — the vendor consoles each review covers) · Risk Management Policy (R-13 — Sami-unavailable risk).

1. Purpose

Turn "who has admin on what" from a memory item into a scheduled artifact. The review IS the compliance object — a periodic, dated, signed record that access is currently what we intend it to be, not what it accumulated to.

2. Scope

Every system listed in Access Control Policy §6 plus every vendor listed in Vendor Management Policy §3. Reviews are per-system with a fixed cadence and a documented procedure — not a monthly meeting where we forget half the systems.

3. Review cadence

Frequency scaled by blast radius of unauthorized access.

System	Cadence	Rationale
Issuer signing seed access (1Password entry + paper backup + Render env var)	Quarterly	Root of paid-tier trust. Any additional holder = potential leak surface.
Render team members + role	Quarterly	Render admin = production env vars = every secret we hold.
Stripe team members + role	Quarterly	Stripe admin = payments + subscription mutation authority.
VPS 157.230.177.45 SSH keys (~/.ssh/authorized_keys for sami and root)	Quarterly	Any SSH key here = installer/tarball mirror mutation authority.
GitHub org (patl4588) owners + admins + repo write	Quarterly	Any org owner = push to main = deploy authority.
1Password shared vault members	Annually	Vault is E2E encrypted; unauthorized addition is bounded (they'd still need master passphrase to read).
Cloudflare DNS admin	Annually	Lower blast (DNS change is visible externally quickly).
Anthropic API key holders (Probo dev key + any prod key)	Annually	Cost surface, not customer-data surface.
DigitalOcean team + billing admin	Annually	VPS lives here; SSH review above catches the more urgent surface.

Reviews happen in the FIRST calendar month of the quarter (Jan/Apr/Jul/Oct) unless a review-trigger event fires first (§5).

4. Review procedure

For each system, the review is a 5-step checklist. Written so it can be done in one sitting; typical duration 15–30 min per system for the quarterly ones.

- List every current holder + role.** From the system's own admin console (Render team page, Stripe team page, cat ~/.ssh/authorized_keys on the VPS, GitHub org people page).
- Compare against intended holders** (Access Control Policy §6, Vendor Management Policy §3). Any mismatch = finding.

3. **Justify every holder still needed.** For each name: one sentence — why do they still need this access? "Sami is the Security Officer" is a valid justification. "I don't remember why we added them" is a finding.
4. **Revoke anything not justified.** Same session — don't defer. If revocation is politically awkward (a former contractor), do it anyway and communicate after.
5. **Log the review.** Timestamped entry with per-holder decisions. See §6 for template.

5. Review-trigger events (out-of-cycle reviews)

The scheduled cadence catches drift. These events trigger an immediate review of the affected system, regardless of scheduled date:

- **Any incident of SEV-2 or higher touching that system's credentials** — per Incident Response Policy §7.5, §7.6.
- **Person leaves the team.** Every access they had is revoked before they walk out the door (or, in a fully-remote team, before the day ends).
- **Person's role changes.** Access is re-scoped to the new role, old access removed the same day.
- **Vendor pricing/policy change** that alters who NEEDS access (e.g., Render team-size upgrade).
- **New root-tier secret added** (e.g., issuer key rotation is a trigger for the "issuer signing seed access" review).
- **Any admin console shows a login by a name we don't recognize** — treat as SEV-1 until proven otherwise, review is one of the response actions.

6. Review log template

Kept as append-only markdown in `~/access-reviews/YYYY-QN-<system>.md` on Sami's box. Each review is one file; the file IS the artifact HIPAA + SOC 2 examiners ask to see.

```
# Access Review - <system> - YYYY QN

**Date:** YYYY-MM-DD
**Reviewer:** <name, role>
**Cadence trigger:** scheduled Q1 2026 / triggered by <event>

## Holders + decisions

| Name | Role | Access since | Still needed? | Decision | Rationale |
|---|---|---|---|---|---|
| Sami Ben Chaalia | admin | 2025-XX | yes | keep | Security Officer per Access Control Policy §6 |
| Pat Linden | owner | 2025-XX | yes | keep | Business owner; operational continuity per BC/DR §5.7 |
| (example) John X | contributor | 2026-05 | no | REVOKE | Contract ended 2026-06; access should have been remove

## Findings

- Any holder not intended per policy: describe + action taken.
- Any policy update needed: describe + link to updated Probo entry.

## Sign-off

Reviewer: _____ Date: _____
```

Every review file gets a companion Probo entry via `addEvidence` (or attached to the corresponding control in the SOC 2 framework) so the compliance program can trace back to the actual review artifact.

7. Findings + remediation

Any finding from §4 step 4 or §5 that isn't remediated during the review session becomes a tracked item:

- Logged as a risk in Risk Management Policy §6 (usually LOW score unless the finding indicates active compromise).

- Owner assigned; target-date within 5 business days for a MEDIUM+ finding, next review for LOW.
- Reviewed at the following quarter's review for that system.

8. Emergency revocation

Any credential holder can revoke ANOTHER holder's access without waiting for a review if there's reasonable suspicion of compromise or misuse. Log the revocation as a SEV-2 incident and initiate a review the same day. The cost of a bad revocation (short-lived lockout, easily restored) is far less than the cost of unauthorized action.

9. Automation

Where feasible, the "list holders" step (§4 step 1) is automated:

- **VPS SSH keys:** `ssh sami@157.230.177.45 'cat ~/.ssh/authorized_keys /root/.ssh/authorized_keys'` — trivially scriptable, no admin console click needed.
- **GitHub org members:** `gh api orgs/patl4588/members` returns the list; `gh api orgs/patl4588/teams/<team>/members` for role-scoped views.
- **Render team:** requires the Render dashboard (no public API for team members at time of writing).
- **Stripe team:** dashboard-only currently.

The scripts for the automatable ones live in `railcall-core/tools/access-review/` (TO BE CREATED). Automation is an accelerator, not a substitute for the human review — the "still needed?" judgement is not scriptable.

10. Review

This policy itself is reviewed annually + any time the review cadence in §3 changes. Cadence tightening (e.g., moving from quarterly to monthly for a specific system) triggers an immediate one-time review of that system to establish a new baseline.

11. Related documents

- **Access Control Policy §6** — the systems inventory that §3 mirrors.
 - **Vendor Management Policy §3** — the vendor console list §3 also covers.
 - **Risk Management Policy §6 R-13** — the "Sami unavailable" risk this policy partially mitigates by ensuring Pat's continued VPS/Render access.
 - **Incident Response Policy §7.5–7.6** — the credential compromise runbooks that trigger §5 out-of-cycle reviews.
-

Acceptable Use Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC1.1 (Integrity + ethical values) · CC1.4 (Attracts + develops competent people) · ISO 27001 A.5.10 (Acceptable use of information) **Companion documents:** Access Control Policy · Password + Credential Policy · Confidentiality Policy · `feedback_railcall_culture.md` (the internal cultural memory this policy codifies).

1. Purpose

State the standard of conduct expected of every person with access to RailCall systems, customer data, or third-party accounts held in RailCall's name. Deliberately short — a two-person team doesn't need a novel, but the standard still has to be written down for it to be enforceable.

2. Scope

Every operator with any credential in the systems listed in Access Control Policy §6 or Vendor Management Policy §3. Currently: Sami and Pat. New operators sign this policy (electronic acknowledgement) as part of access provisioning.

3. The core rules

3.1 No fake-green

No commit, no policy, no customer-facing claim shall assert something we cannot demonstrate on the record. Every "IMPLEMENTED" cites the code. Every "in progress" names an owner + a date. Every gap is a gap, not a euphemism.

This is the load-bearing rule. Every other rule follows from it.

Applies to: commit messages, docs, marketing pages, investor materials, compliance policies, incident notes, customer emails.

3.2 Evidence before prose

Read the code before you write the finding. Read the log before you write the post-mortem. If the finding can't cite a source, the finding is wrong.

Rewriting a finding to match the code after the fact is fine and expected. Writing a finding first and then bending the description to protect it is falsification.

3.3 Secrets never in text

No API key, seed hex, password, or token ends up in email, Slack, chat, git, issue tracker, or any surface an attacker with search access can trawl. Signal disappearing messages + 1Password shared vault + `~/*key*.txt` at 0600 are the accepted transports.

If a secret DOES land in a durable channel by accident: rotate it within 4 hours (per Incident Response Policy §3 SEV-2 response window), log the incident, do the post-mortem.

Exception: dev/test keys explicitly scoped to a throwaway resource, when the resource is documented as throwaway. Even then: rotate on any doubt.

3.4 Dry-run by default

Any command touching production state defaults to dry-run when the flag exists. Only strip the dry-run when the plan has been read.

Applies to: station releases (leak-gate + fresh-install smoke run before pin change), `install.sh` re-pins (byte-compare before commit), Render env-var changes (single-variable edits, never bulk paste), Stripe operations (test first when the API supports it), any bash `rm -rf` (list before delete).

3.5 One SoT per fact

Every fact lives in one authoritative place. When a fact changes there, every downstream copy updates in the same change. Downstream copies drift; SoT never does.

Concrete SoTs: - **Source code:** `github.com/patl4588/railcall-core|engine|contrib`. Not local checkouts, not release tarballs. - **Compliance artifacts:** `railcall-core/compliance/` in git. Probo is a viewer, not the SoT. - **Customer entitlements:** Render Postgres. Local install caches are derived state. - **Issuer signing seed:** Sami's 1Password + paper backup. Every other copy is either the ceremony's ephemeral output (shred after use) or an env-var copy set from the SoT. - **This policy set:** git. See `compliance/policies/README.md`.

3.6 Least privilege by default

Every new credential is scoped as narrowly as the task allows. Restricted keys preferred over standard keys. Read-only preferred over write. When "just this once" pressure argues for a broader scope, log the decision in the change record so future review can audit it.

3.7 No unilateral destruction of shared state

`git push --force` to `main`, `git reset --hard` on a shared branch, dropping a production table, deleting a Stripe customer, revoking another operator's access — none happen without either (a) written acknowledgement from another operator, or (b) an active incident where the operator is the IC and logs the action in the incident note.

Local destruction (your own branch, your own checkout, your own machine) is your call. Shared destruction is a two-person decision.

3.8 Honest communication with customers + investors + regulators

Same rule as §3.1 externally. If a customer asks whether we're SOC 2 compliant and we're not: we say "not yet, targeting Q4 2026, here's the readiness posture." Not "we're in progress" (which reads as "will be soon"). Not "SOC 2 controls implemented" (which reads as "the audit is done").

4. Prohibited uses

Rare that these need saying but stating them makes enforcement cleaner:

- **Personal use of customer data.** Zero. Not for demo videos, not for screenshots, not for tests.
- **Personal use of RailCall infrastructure.** Rendering personal websites on the VPS, running personal databases on Render, using the Stripe account for personal invoices. All out.
- **Using operator credentials for anything outside their scope.** Sami's GitHub PAT is for RailCall repos. Pat's Render access is for RailCall services. Not for side projects.
- **Sharing credentials.** Ever. Not even with each other. Access is granted per-person via the account's own team/collab mechanism.
- **Bypassing controls to "just get it done."** Skipping tests to ship faster. Disabling the leak gate to publish a tarball. Answering a compliance question with what the customer wants to hear instead of what's true. All violate this policy AND every other policy in the set.

5. Escalation

If you see any of the following, escalate immediately (Signal call to the other operator + note in `~/incidents/`):

- Another operator violating §3 or §4.
- Yourself under time pressure to violate §3 or §4.
- A tool or automation about to violate §3 or §4 (an AI agent making a bad commit, a script about to `git push --force`).

Escalation is not blame; escalation is the safety net. The cost of a false escalation is a Signal call. The cost of a missed one is a preventable incident.

6. Enforcement

- **First violation:** discussed, documented, corrective action taken. No formal sanction — the person meant well, the process failed.
- **Repeated violation of the same rule:** access to the affected system scoped down + a written plan for how the process changes to prevent recurrence.
- **Deliberate violation with knowledge:** access revoked, considered grounds for removal from the team.

Team of two operating under trust: the enforcement mechanism here is the relationship, not a formal HR process. When the team grows, this section tightens.

7. Acknowledgement

Every operator signs (electronic acknowledgement in Probo, or paper) at onboarding and re-signs at each major revision of this policy. Sami's adoption of this v1 is recorded in Probo document `Access Control Policy` §8 and mirrored to `compliance/policies/README.md`.

8. Related documents

- Access Control Policy §6 — the systems this policy governs conduct within.
 - Password + Credential Policy — the per-credential hygiene that §3.3 relies on.
 - Confidentiality Policy — the customer-data-specific extension of §4.
 - Change Management Policy §6 — the standing merge authorization + emergency change path that §3.6 (least privilege) interacts with.
 - `feedback_railcall_culture.md` (Claude auto-memory) — the running record of cultural rules this policy codifies.
-

Confidentiality Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 C1.1 (Confidentiality — protects info designated as such) · C1.2 (Disposes of confidential info) · HIPAA §164.312 (safeguards) · GDPR Art. 5(1)(f) integrity + confidentiality **Companion documents:** Data Retention & Deletion Policy · Encryption Policy · Access Control Policy · Acceptable Use Policy · Vendor Management Policy · railcall.ai/legal/privacy (customer-facing companion).

1. Purpose

Define what RailCall treats as confidential, how it's handled, and what happens when confidentiality is breached. Written as the operator-internal companion to the customer-facing terms + privacy policy.

2. Classification

Every piece of information touched by RailCall falls into exactly one class. The class determines encryption, access, retention, and disclosure rules.

Class	Definition	Examples	Default handling
PUBLIC	Intended for open distribution	railcall.ai marketing content, published docs, station tarballs on GitHub Releases, this SRA + policy set post-adoption, issuer public key	No encryption required; standard hosting integrity controls apply (checksum + TLS in transit)
INTERNAL	Not for external distribution; leak wouldn't cause direct harm	Draft policies (like this one), incident notes, internal decisions, cost analyses, Probo state, this policy folder before adoption	0600 file perms, git-hosted, no automated external egress
CONFIDENTIAL	External distribution would harm customer or business relationship	Customer email addresses, customer usage metadata, signed BAA texts, Stripe customer records, restricted API tokens, DPA-scoped agreements	Encrypted at rest (Render Postgres managed encryption or vault AES-256-GCM), 0600 files, access via Access Control Policy §6 only
SECRET	Compromise = existential or trust-root failure	Issuer signing seed, individual install signing seeds, Stripe standard secret keys, RailCall_ISSUER_SEED, VPS root SSH key	OS keychain or 1Password only; never written to any file we don't control; rotation runbook in Encryption Policy §7

Anything not obviously PUBLIC is INTERNAL until re-classified. Anything customer-identifying is CONFIDENTIAL. Anything that signs, mints, or grants is SECRET.

3. What we treat as customer confidential

- **Identifying info:** email address, org name, Stripe customer id, IP addresses collected in Render logs.
- **Payment info:** we never see full card numbers (Stripe handles); we DO see subscription state, invoice history, plan, seat count.
- **Usage patterns:** any timing, frequency, or content signal derivable from meter events (`sha256(api_key)` , timestamps, run counts).
- **Configuration:** which integrations they have configured, at what BYOK count (though we never see the credentials themselves — see §5).
- **Incident-related detail:** anything a customer told us during triage.

We do NOT treat as confidential (it's their choice to publish or not): - The customer's decision to be our customer (unless they explicitly ask us not to reference them). - Aggregate + de-identified statistics ("we have N paying customers", "median seat count is X").

4. What we treat as our own confidential

- **INTERNAL:** business plans, revenue figures, unadopted policies, internal team communications, customer names in aggregate stats when not yet redacted.
- **CONFIDENTIAL:** contracts with vendors, individual employee compensation (N/A for a two-person team but stated for future headcount), unshipped code that hasn't been released, competitive analysis.
- **SECRET:** the issuer seed (§164.312 trust root), production credentials for every vendor in Vendor Management §3, private keys of any kind.

5. What we architecturally do NOT hold

Stated explicitly because absence of storage is the strongest confidentiality control:

- **Customer workflow inputs + outputs.** The engine runs on the customer's box. We physically don't have this data.
- **Customer PHI or PII of end-users.** Blocked at construction by `phi_guard` (Encryption Policy §5).
- **Customer BYOK secrets.** They live in the customer's 0600 vault on their machine. We never see the values — only that a credential of a given shape exists (integration name + a hash).
- **Customer workflow structure post-execution.** Receipts are integrity hashes over redacted metadata; the actual workflow topology + parameter values stay on the customer's box.

Consequence: for most customer deployments, RailCall is not a business associate under HIPAA (§160.103) because we do not create, receive, maintain, or transmit PHI on their behalf. See SRA §1 for the full analysis.

6. Handling by class

6.1 PUBLIC

- Distributable freely.
- Integrity matters (customers rely on it) — signed release tarballs, checksum-pinned installer.
- No encryption in transit required, but we use TLS by default anyway.

6.2 INTERNAL

- Stored in git (source code + policies + docs) or in Sami's personal channels (Signal, personal 1Password vault entries for INTERNAL work notes).
- Not shared externally without explicit consideration.
- No customer identifiers appear in INTERNAL artifacts unless the artifact is directly for that customer.

6.3 CONFIDENTIAL

- Encrypted at rest: Render managed Postgres for gateway DB rows; 0600 vault for local storage.
- Access limited per Access Control Policy §6.
- Never appears in logs, error messages, or third-party analytics.
- Shared externally only under a signed agreement (NDA, DPA, BAA) matching the customer relationship.

6.4 SECRET

- Storage per Encryption Policy §4 (OS keychain, 1Password, Render env vars marked `sync:false`).
- Never leaves the environment it's authorized for. The issuer seed lives in Render + Sami's 1Password + Sami's paper backup — nowhere else.
- Rotation runbook per Encryption Policy §7 in case of loss OR suspected compromise.

7. Disclosure

We disclose confidential information externally only when:

1. **Legally required** (subpoena, court order, regulatory production request). Sami consults counsel before responding; where legally permitted, we notify the affected customer before disclosing.
2. **Contractually required** (a customer's BAA-mandated breach notification; a Stripe-mandated fraud investigation cooperation).
3. **Necessary for the customer's own operation** (customer's engineering lead asks for their subscription-state — we verify identity and provide).
4. **Explicitly authorized by the affected party** (customer says "yes, use us as a case study").

We never disclose to:

- Marketing partners without opt-in.
- Analytics providers without opt-in and de-identification.
- Other customers.
- Investors or advisors, beyond de-identified aggregates.
- Search-indexable channels of any kind.

8. NDAs

- **Inbound (customer sends us their info under NDA)**: we honor it. NDA scope becomes part of that customer's CONFIDENTIAL surface until the NDA lapses.
- **Outbound (we send info under NDA)**: default templates in `legal/nda-mutual.md` (TO BE CREATED). Standard mutual NDA — 3-year confidentiality window, standard carveouts for publicly-available info + independently-developed info + legally-required disclosure.
- **NDAs are tracked in Probo** (per-vendor entry in the risk register + document link).

9. When confidentiality fails

Cross-reference Incident Response Policy §7 for compromise runbooks. Specifically:

- **PUBLIC integrity failure** (someone tampered with a release tarball on GitHub): the byte-compare in the release ceremony catches this pre-pin. If it slips past: SEV-1, see Change Mgmt §8 + Incident Response §7.7.
- **INTERNAL leak** (draft policy accidentally posted): re-classify what was leaked; assess whether it matters (usually low); update policy if the leak points at a genuine confidentiality process failure.
- **CONFIDENTIAL leak** (customer identifier or usage pattern exposed): SEV-2. Notify affected customer per breach-notification windows in §10 below.
- **SECRET leak** (trust root or production credential): SEV-1. See Incident Response §7.1 (issuer seed) or §7.5 (Stripe key). Rotation runbook + all-customer notification.

10. Breach notification

Windows sync with Incident Response Policy §3 + BAA_DRAFT.md §3.3:

- **Customers with an active BAA**: within 10 days of discovery.
- **Regulators for HIPAA breaches ≥ 500 individuals**: within 60 days per §164.408 (HHS OCR).
- **GDPR supervisory authority for EU personal data breaches**: within 72 hours per Art. 33.
- **State AGs**: varies; assume 30 days as safe default.
- **Public disclosure**: SO decides; default proactive if any customer is affected.

11. Post-employment / post-relationship

- **Operators leaving the team:** all confidential material returned or destroyed (SSH key removal, 1Password vault removal, Render/Stripe team removal per Access Review Policy §5). NDA remains in effect for its stated term.
- **Vendors we stop using:** per Vendor Management Policy §5 offboarding — cut access, extract data, confirm deletion, remove from Vendor Mgmt §3 + BAA Exhibit C.
- **Customers who leave:** account marked inactive per Data Retention Policy §4.1; personal data may be erased on request per §4.2; compliance-retention data kept for the 6-year HIPAA window even after account close.

12. Review

At each station release + at least annually. Classification changes (something INTERNAL becoming CONFIDENTIAL because a customer requested it be treated as such) trigger an immediate re-application of §6 rules to that item.

13. Related documents

- **Data Retention & Deletion Policy** — the retention windows §11 references.
 - **Encryption Policy** — the algorithms + key lifecycles that make §6 rules technically enforceable.
 - **Access Control Policy** — who has the credentials that reach each class.
 - **Acceptable Use Policy §3.3, §4** — the operator-conduct rules that keep §6 from being violated.
 - **Incident Response Policy §3, §7** — the response process for §9 confidentiality failures.
 - **BAA_DRAFT.md** — the customer-facing legal instrument this policy operationalizes.
 - **railcall.ai/legal/privacy** — the customer-facing companion to §3.
-

Software Development Lifecycle Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC7.1 (System changes designed with security) · CC8.1 (Change management) · NIST SSDF (Secure Software Development Framework) · OWASP ASVS L1 baseline **Companion documents:** Change Management Policy · Access Control Policy · Confidentiality Policy · Acceptable Use Policy.

1. Purpose

Codify how RailCall code is actually written, reviewed, tested, released, and maintained. Reflects the AI-assisted workflow (Claude Code + human review) that produces most of the codebase in 2026, because ignoring that would make the policy fiction.

2. Scope

Every code repository owned by the RailCall project: - `railcall-core` (public — CLI, gateway, install scripts, compliance) - `railcall-engine` (sealed — local runtime, sealed IP boundary per `project_railcall.md`) - `railcall-contrib` (public — website, docs, mirror) - Any future repository under `github.com/patl4588`.

3. Development principles

- **Code with intent.** Every file, function, and commit exists because there's an invariant or gap it addresses. If it can't be named, it doesn't ship.
 - **Read before write.** New code is written against knowledge of the existing code, not against guesses. AI-authored code is verified against the file it touches, not against the model's guess of what the file contains.
 - **Tests are the invariant record.** Every test asserts a property we care about surviving. Tests are the primary artifact — code is what makes tests pass.
 - **Small, reversible commits.** A commit does one thing, cites its evidence, and is safely revertible. Multi-purpose commits are refactored before push.
 - **Honest naming.** Function names, variable names, module names describe what the thing does, not what we wish it did. `signing_seed_status()` returns the seed's rest posture; it does not return the seed.
-

4. The workflow

For most changes:

1. **Frame the change.** State the invariant being defended or the gap being closed in one sentence. If Claude Code is authoring, this is the prompt.
2. **Read the touchpoint files.** Not a summary — the actual bytes on disk today. AI agents that skip this step regularly produce plausible-but-wrong output.
3. **Write the change + at least one test.** Test must fail pre-change and pass post-change. This is what makes it a test rather than an assertion of "the code I just wrote does what I just wrote."
4. **Run the local suite** for anything the change could plausibly affect (not just the file touched — check callers).
5. **Commit with an evidence-forward message** — WHY, not just WHAT. Cite tests + affected surfaces.
6. **Push.** Per Change Management Policy §4, this triggers deploy for `railcall-core:main`.
7. **Verify post-deploy** for any Enhancement or Breaking change.

For AI-assisted commits specifically:

- **The human reviewer is Sami.** Not "the AI reviewed itself"; not "the tests passed so we're done." A human eye looks at every AI-generated commit before push (unless it falls under the standing merge authorization + is Standard-class per Change

Management §6).

- **The AI cites evidence in commit messages** — file:line references, test names, invariants defended. Prose without citation reads as marketing and gets rewritten.
- **Suspected hallucinations get verified.** If Claude claims a function exists, grep for it before believing.

5. Secure coding standards

Enforced by convention + code review + the specific technical controls listed below. This is not an exhaustive OWASP list; it's the set that has actually mattered in this codebase.

5.1 Never log secrets

- No API keys, seeds, tokens, or passwords appear in any log statement.
- Vault operations log key NAMES + operation type, never values.
- Audit chain scrubs `email` and any field on the credentials + PHI lists (`P0-4` remediation).
- Every third-party call that could echo a secret in an error is wrapped to catch + redact.

5.2 Fail closed

- Every gate refuses on unknown input. `/v1/seat/checkin` returns 401 on unknown hash; issuer mint returns 503 if seed unset; entitlement verify returns free-tier on any tamper.
- Absence of configuration = refusal, never permission. `RAILCALL_ISSUER_SEED` unset means 503, not "sign anyway with a random seed."

5.3 Idempotency on external effects

- Every write that COULD be retried (webhook, meter, checkout provision) is dedup'd on a scoped nonce. Duplicate delivery is impossible in practice + provable in test.

5.4 Input validation before DB touch

- Every endpoint validates shape + bounds before opening a DB connection. 400 for malformed, 401/403 for unauthorized, 503 for unconfigured. 500 is a genuine internal error only.

5.5 No SQL string interpolation

- Every query goes through parameterized `?` placeholders + `ph()` helper for Postgres/SQLite portability. No f-string SQL. Ever.

5.6 Cryptographic algorithms per Encryption Policy §3

- Only algorithms listed in Encryption Policy §3 are used. Adding a new one requires updating that policy first.

5.7 No hardcoded secrets in git

- `.gitignore` excludes `.env`, `keys.local.json`, `*seed*.txt`, `*key*.pem`.
- Release script's leak gate refuses tarballs containing recognizable secret shapes.
- Pre-commit review catches accidents.
- If something slips: rotate immediately (Incident Response §7), then post-mortem the process failure.

5.8 Explicit typing on public surfaces

- FastAPI endpoint parameters typed with `Form(...)` / `Body(...)` etc. Type mismatch = 422 automatic.
- Python type hints on public-API functions; internal helpers may skip.

6. Test discipline

- **Unit tests** for logic; integration tests for cross-module flows; e2e tests for the full user path.
- **Named suites** per surface: `test_seat_checkin.py`, `test_stripe_lifecycle.py`, `test_activate_e2e.py`, `test_paid_full_e2e.py`, `test_cli_seed_store.py`, `test_licensing_endpoints.py`, `test_stripe_seat_wiring.py`, `test_entitlement.py` (engine), etc.
- **Every P0 remediation has a dedicated test file.** `phi_guard.py` → `test_phi_guard.py` (29 checks). `audit_chain.py` → `test_audit_chain.py` (17). `seed_store.py` → `test_seed_store.py` (16) + `test_cli_seed_store.py` (8).
- **Tests assert LIMITS as well as wins.** `test_audit_chain.py:4a` asserts tail-truncation is undetectable — nobody can accidentally over-claim later.
- **Tests run against real behavior** — `test_paid_full_e2e.py` spawns a real gateway on loopback and drives the real CLI code path, no mocks on either side of the wire.
- **Fake-green is caught.** Test suites report skipped tests loudly, not silently. A silently-skipped critical assertion is worse than a red test.

7. Dependency management

- Python dependencies pinned in `requirements.txt` (railcall-core). Engine dependencies are minimal + stdlib-first by convention.
- Adding a dependency requires: 1. Justification (why we need this vs. writing it). 2. License compatibility check. 3. Security posture check (maintained? recent CVEs? source-available?). 4. Vendor Management Policy §4 onboarding if the dep is a network service (unlike a Python library).
- Removing a dependency is preferable to updating a compromised one when the removal is feasible.

8. Vulnerability response

- **Dependency CVE reported:** if the package is in our path, pin the fixed version + re-run every test suite + retrospect any station releases that might have shipped the vulnerable version (Incident Response §7.7).
- **In-house vulnerability reported:** by external researcher or internal review — treat per Incident Response §3 severity classification. `security@railcall.ai` is the intake channel (documented in `railcall-contrib/website-v2/app/legal/security`).

9. Sealed engine boundary

Per `project_railcall.md` (memory), `railcall-engine` is the sealed-IP repo. Never surface engine internals in:

- Public repos (`railcall-core`, `railcall-contrib`).
- Public station releases (the release script's leak gate enforces this — refuses tarballs containing engine-specific factory/moat artifacts like `workflow_library/`, `combinatorics_index.json`, `architecture_*`, etc.).
- Third-party pushes (never PyPI-publish engine internals; see MCP.md deliberate decision to CLI-command MCP instead of PyPI-package it, in `MCP.md` and `project-paid-tier` memory).

10. Code review

- **Sami reviews Claude-authored commits.** For Standard-class changes under the standing merge authorization, review may be post-hoc. For Enhancement + Breaking, review is pre-push.
- **Human-authored commits** get the same review, mutually. In a two-person team, review is often synchronous over Signal.
- **Review focus:** correctness > readability > style. If it's incorrect but pretty, it doesn't ship.

11. Documentation

- **Code:** minimal comments; identifiers describe what the code does. Comments are for WHY it's non-obvious (hidden constraint, subtle invariant, workaround, surprising behavior).
- **Public APIs:** documented on the endpoint or CLI command itself, not in a separate file that drifts.
- **Compliance:** this policy set + the SRA + the BAA draft. See `compliance/policies/README.md` for the index.

12. Third-party service integrations (BYOK)

When we add a new integration to the station (Slack, Airtable, GitHub, etc.):

1. Design + implement locally with the customer's own credential + no data leaving their box.
2. Add to the registry (`integration_registry.py`) with proper `action_class` per the effect-node classification (see canvas + workflow_engine docs).
3. PHI-guard the payload path — every integration must go through `approval_airlock.py:redact()`.
4. Test with a fixture + a real cred (dev-scoped).
5. Update `Vendor Management Policy §3` only if we ourselves rely on the vendor for hosted service — a BYOK integration is customer-scoped and doesn't create a subprocessor relationship.

13. Release + deploy

Per Change Management Policy §5 (station release ceremony). This policy doesn't repeat the steps; it points at them.

Deploys of the gateway happen automatically on push to `railcall-core:main`. Not automatic: station releases (require the full ceremony) or VPS website rebuilds (require `pm2 restart` after a `public/` change).

14. Review

At each station release + at least annually. Any new secure-coding rule added to §5 requires an example of the class of failure it prevents, so future-us knows why it exists.

15. Related documents

- **Change Management Policy** §4, §5 — the deploy pipeline this SDLC feeds.
 - **Access Control Policy** §6 — the credential surface development touches.
 - **Confidentiality Policy** §5 — the "what we do NOT hold" invariants coding is bound to preserve.
 - **Acceptable Use Policy** §3 — the cultural rules (no fake-green, evidence before prose) that this policy operationalizes.
 - **Incident Response Policy** §7.7 — the dependency-compromise runbook §8 refers to.
-

Password + Credential Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.1 (Logical access) · CC6.2 (New user provisioning + deprovisioning) · HIPAA §164.308(a)(5)(ii)(D) password management · NIST SP 800-63B (Authenticator standards) **Companion documents:** Access Control Policy · Access Review Policy · Confidentiality Policy · Encryption Policy §4.

1. Purpose

State the human-side hygiene for every credential a RailCall operator holds. Complements Access Control Policy §6 (which lists WHERE access lives) with rules about HOW the credentials to that access are handled.

2. Scope

Every credential an operator uses to reach a system in Access Control Policy §6 or Vendor Management Policy §3. Covers passwords, SSH keys, API keys, OAuth tokens, TOTP secrets, hardware token seeds — anything that authenticates a person.

Out of scope: agent-to-system credentials (Ed25519 install seeds, blind api_key hashes) — those are covered in Encryption Policy §4.

3. The rules

3.1 Password manager is mandatory

1Password is the RailCall-standard password manager. Every operator uses it for:

- Every login the operator holds for RailCall-related systems.
- Every API key the operator generates for personal use of a RailCall-related service.
- Every SSH private key file (encrypted 1Password secret note, not just filesystem 0600).
- The issuer signing seed (see Encryption Policy §4.2 for the full ceremony).

Rationale: browser-cached passwords + local files + reused mental passwords each fail differently. 1Password consolidates the failure modes into one (the master passphrase), which is the failure mode we can most reliably defend.

3.2 Master passphrase requirements

- **Length $\geq$ 20 characters OR $\geq$ 5 words** if a passphrase.
- **Not a variation of any password used elsewhere.** Ever.
- **Not written down anywhere except:** one sealed paper copy in Sami's physical safe (offline recovery backup only).
- **Not typed in front of a camera** — Zoom, Loom, screen shares.
- **Rotated on any suspicion of exposure.** Not on a schedule (per NIST 800-63B guidance — rotation without cause weakens more than it strengthens).

3.3 MFA everywhere

Every account that supports MFA has it enabled. No exceptions for "just this one dev thing" — that's exactly the account that gets compromised.

Preferred MFA methods, ranked:

1. **Hardware token** (YubiKey via WebAuthn) — bind to the physical device the operator carries.
2. **TOTP in 1Password** (or a dedicated authenticator app) — the code is derived from a secret 1Password holds, so losing the phone doesn't lose access as long as the 1Password vault is intact.
3. **TOTP in Authy or Google Authenticator** — acceptable but weaker (single-device by default).

4. **SMS MFA** — NOT acceptable except as a fallback for services that offer nothing else. SIM-swap attacks are the classic bypass.
5. **Email MFA** — same as SMS; acceptable only when the target service offers nothing else.

Enabled today (baseline to maintain):

- GitHub org: MFA required for org members.
- Render: MFA on Sami's account; enforce for team on quarterly access review.
- Stripe: MFA required for all team members.
- 1Password: MFA required (device authorization + TOTP).
- Anthropic Console: MFA on Sami's account.
- DigitalOcean: MFA on Sami's account.
- Cloudflare: MFA required.

3.4 No password reuse

Each service gets a unique, generated password. 1Password's generator produces them; operators do not invent their own.

If the same password appears twice across services in the vault: treat as an incident (SEV-3), rotate both immediately.

3.5 API keys — scoped, dated, rotated

Every API key issued to an operator (Stripe restricted key, Anthropic key, GitHub PAT, etc.):

- **Named** for its purpose, dated by creation. `"RailCall - Stripe price setup - 2026-07-21"`.
- **Scoped** to the minimum permissions the task requires. Restricted keys > standard keys.
- **Rotated** on any suspicion of exposure, when the task ends, or annually — whichever first.
- **Revoked** immediately when the task ends if it was a task-scoped key.
- **Never checked into git.** `.gitignore` guards; leak gate in station release catches secondary paths.

Standing keys (persist beyond one task):

- Render `RAILCALL_ISSUER_SEED` — the trust root; rotation ceremony per Encryption Policy §7.
- Stripe standard secret key — rotates only on suspicion of compromise; standing because it's how the webhook + provisioning endpoint work.
- Anthropic API key for Probo — dev-scoped, rotates on operator change.

Task-scoped keys expected lifespan: hours to days. Standing keys: months to years.

3.6 SSH keys

- **Ed25519 preferred** (`ssh-keygen -t ed25519 -C "operator@date"`).
- **Passphrase-protected.** No naked private keys on disk. Passphrase stored in 1Password, entered via ssh-agent.
- **One key per operator per purpose.** Sami's VPS key (`~/.ssh/id_ed255199`) is separate from Sami's GitHub key, separate from Sami's server admin key.
- **Rotation:** on operator device change, on suspicion of compromise, or annually — whichever first.
- **Revocation:** remove from every `authorized_keys` file the key was granted to. Access Review Policy §4.1 catches drift.

3.7 OAuth tokens + third-party session tokens

- Sessions to admin consoles (Render, Stripe, GitHub) expire per each service's default; operators do not extend session lifetime.
- OAuth tokens issued by any service that supports scoped issuance get minimum scope.
- Bookmarks + saved sessions on the operator's browser respect 1Password's autofill — the operator does NOT save the login in the browser's own manager (Chrome/Safari/Firefox password managers are OFF-limits for RailCall credentials).

3.8 Recovery + fallback

Every credential has a defined recovery path:

- **1Password vault:** master passphrase + emergency kit (paper copy in Sami's safe). Recovery from device loss requires the emergency kit.
- **MFA device:** each service's backup codes stored in 1Password.
- **SSH keys:** paper copy of the recovery instructions (which servers, which `authorized_keys` files) in Sami's safe. NOT paper copy of the private key — the paper is instructions to regenerate + re-authorize.
- **Issuer signing seed:** the seed itself is in Sami's paper safe + 1Password. This is the ONE credential we back up in its raw form because losing it is existential (see BC/DR Policy §5.1).

Every recovery path is tested at least once per year (annual issuer-seed rotation exercises the SEED path; scheduled password-manager restore drills exercise the MFA + SSH paths).

4. Provisioning + deprovisioning

- **New operator joins the team:** 1. 1Password shared vault invite. 2. GitHub org invite + MFA verification. 3. Render team invite + MFA verification. 4. Stripe team invite + MFA verification. 5. VPS SSH key added to appropriate `authorized_keys`. 6. Access Control Policy + Acceptable Use Policy signed (Probo). 7. Read this policy + acknowledge. 8. Documented in Access Control Policy §6.
- **Operator leaves:** 1. Every credential revoked (per Access Review Policy §5). 2. 1Password shared vault access removed. 3. Personal 1Password vault stays (their property). 4. NDA remains in effect for its term per Confidentiality Policy §11. 5. Documented in Access Control Policy §6 with revocation date.

5. Prohibited practices

Restating the sharp lines:

- **Sharing credentials.** No, not even between Sami and Pat. Each operator uses their own account with their own MFA.
- **Reusing personal passwords for work.** No — 1Password removes the excuse.
- **Storing credentials in text files, Signal messages, or GitHub issues.** No, ever. If a credential lands in a text channel by accident (§Acceptable Use §3.3): rotate + incident.
- **Turning off MFA "just for this session".** No. If MFA is broken, fix MFA, don't disable it.
- **Skipping 1Password because "this one is fine to remember".** No — the exception becomes the norm within a quarter.

6. Review

- **Per-credential:** at the cadence in Access Review Policy §3 (mostly quarterly for high-blast, annually for lower).
- **This policy:** at each station release + at least annually.
- **1Password vault contents:** scanned quarterly for reused passwords + stale entries; monthly for anomalous new entries.

7. Related documents

- **Access Control Policy §6** — the systems this policy's credentials reach.
- **Access Review Policy §3** — the cadence for reviewing who holds each of these credentials.
- **Encryption Policy §4** — the SoT-vs-copy story for the load-bearing secrets (issuer seed, install seed).
- **Acceptable Use Policy §3.3** — the operator-conduct rule that says secrets never go in text.
- **Confidentiality Policy §2** — the SECRET class that most of the credentials in §3.5 fall into.
- **BC/DR Policy §5.1, §5.8** — the recovery runbooks that assume §3.8 is in force.

Physical Security Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.4 (Restricts physical access) · HIPAA §164.310 Physical Safeguards · ISO 27001 A.7 (Physical + environmental security) **Companion documents:** Vendor Management Policy §3 · Access Control Policy · BC/DR Policy §5.8.

1. Purpose

Document RailCall's physical security posture honestly. Short by design — the honest answer for a fully-remote two-person team with no owned physical infrastructure is much shorter than a template borrowed from a company with data centers.

2. Scope

- **Cloud infrastructure** (Render for the gateway, DigitalOcean for the VPS + snapshots, Cloudflare for edge) — physical security is 100% these vendors' responsibility.
- **Operator workstations** (Sami's laptop, Pat's laptop) — physical security is the operator's own responsibility, with baseline requirements below.
- **Physical secret backups** (paper copies of the issuer signing seed + 1Password emergency kit) — kept in Sami's personal safe.

3. RailCall does not operate any physical infrastructure

Stated explicitly so absence of controls cannot be misread as a gap. RailCall does not own or operate:

- Any data center, colocation cage, or server room.
- Any office premises with servers on-prem.
- Any customer-facing physical hardware, appliance, or device.

Every byte of RailCall-controlled infrastructure runs on someone else's physical infrastructure. This is architectural, not accidental — a two-person team with a local-first product doesn't need physical infrastructure to serve the product.

4. Cloud provider physical controls (inherited)

We rely on the following providers' physical security programs. Reviewed annually per Vendor Management Policy §7.

Provider	Physical posture (as published)	Where it applies to us
Render	SOC 2 Type II, hosted on AWS underneath — inherits AWS data-center physical controls (24x7 armed guards, biometrics, media handling, environmental)	Gateway service + managed Postgres
DigitalOcean	SOC 2, SOC 3, ISO 27001 — DC access controls documented in their trust portal	VPS 157.230.177.45 + weekly snapshots
Cloudflare	SOC 2 Type II, ISO 27001 — global edge with physical DC controls per their trust page	DNS + edge cache for railcall.ai
GitHub	Microsoft's SOC/ISO/PCI posture applies	Source code + release artifacts
Stripe	SOC 1/2 Type II, PCI-DSS Level 1	Payments infrastructure
Anthropic	SOC 2 Type II	Probo dev-only LLM calls

Compliance transfer: any customer requiring physical DC certifications gets the answer "we inherit them from our cloud providers; their SOC reports are available under NDA on request; see Vendor Management §3 for the full list."

5. Operator workstation baseline

Every operator laptop must maintain:

- **Full-disk encryption enabled** (macOS FileVault, Windows BitLocker, Linux LUKS). No exceptions. FileVault on Sami's laptop is verified quarterly.
- **Screen lock ≤ 5 min idle timeout** with password/biometric to unlock.
- **OS + browser + relevant apps updated** within 30 days of vendor patch release for security updates.
- **No unattended access in public spaces** — laptop stays with the operator or is physically locked away.
- **Loss/theft = immediate incident** (BC/DR §5.8): remote wipe if the box supports it, credential rotation, 1Password vault re-key.

Not enforced by any device management system today (no MDM at team size 2). Enforced by the operator's own discipline + the credential rotation runbook that makes a stolen laptop's contents rotate to worthless within hours.

6. Physical secret storage

The one exception to §3 — RailCall holds two physical items whose loss would be operationally serious:

- **Issuer signing seed paper backup** — printed on plain paper, sealed in an envelope, stored in Sami's personal fireproof safe. Location known only to Sami; sealed backup instructions (envelope location + safe combination) shared with Pat via 1Password emergency kit.
- **1Password emergency kit** — the recovery-code paper for Sami's 1Password vault. Same safe.

Both are needed for recovery only in scenarios where digital backups are also unrecoverable (§BC/DR §5.7, §5.8). Reviewed annually — verify both still exist + are legible + the safe still opens.

7. Related documents

- **Vendor Management Policy §3** — the cloud providers whose physical controls we inherit.
 - **Access Control Policy §6** — the operator inventory whose devices §5 applies to.
 - **BC/DR Policy §5.7, §5.8** — the recovery scenarios that require §6 physical backups.
 - **Confidentiality Policy §2** — the SECRET class classification of the physical seed backup.
-

Service Availability & Commitments Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 A1.1 (Availability commitments met) · A1.2 (Environmental protections + recovery) · A1.3 (Recovery + change management support availability) **Companion documents:** BC/DR Policy · Incident Response Policy · Vendor Management Policy · railcall.ai/legal/terms (customer-facing companion).

1. Purpose

State what customers can actually expect from RailCall in terms of uptime, degradation, and communication — and what happens when we miss. Written to match reality, not aspiration. Enterprise buyers ask for an SLA; this is the operator-internal version that backs the customer-facing SLA in the Terms of Service.

2. Service classes

RailCall has three service classes with different availability expectations because they have different customer impact profiles.

2.1 Local station (customer's own machine)

- **Availability commitment:** N/A — runs on the customer's box; our uptime doesn't affect it.
- **What we owe:** the station tarball at railcall.ai/railcall_station.tar.gz must be reachable + integrity-verified, so customers can reinstall + get updates.
- **Customer expectation:** if their box is up, the station is up.

2.2 Public installer + tarball mirror (railcall.ai/install.sh + [/railcall_station.tar.gz](https://railcall.ai/railcall_station.tar.gz))

- **Availability commitment:** **99.5% monthly** — roughly 3.5 hours of allowed downtime per month.
- **What we owe:** the URL responds with the pinned installer or tarball, byte-identical to the GitHub Release copy.
- **Recovery target (RTO):** **4 hours** per BC/DR §5.4.
- **Failure mode:** existing customers unaffected; new installs blocked. Communicate via Discord + Twitter.

2.3 Gateway (railcall-core.onrender.com — paid tier only)

- **Availability commitment:** **99.5% monthly** for the paid-tier endpoints ([/v1/entitlement/mint](https://railcall-core.onrender.com/v1/entitlement/mint), [/v1/seat/checkin](https://railcall-core.onrender.com/v1/seat/checkin), [/v1/attestation/countersign](https://railcall-core.onrender.com/v1/attestation/countersign), [/create-seat-checkout-session](https://railcall-core.onrender.com/create-seat-checkout-session), [/v1/webhooks/stripe](https://railcall-core.onrender.com/v1/webhooks/stripe)).
- **What we owe:** endpoints respond within 2 seconds under normal load; return honest failure codes (503 for our-side, 5xx from Render for their-side, per §4).
- **Recovery target (RTO):** **4 hours** per BC/DR §5.2.
- **Failure mode:** paid customers with existing entitlements are unaffected (verification is local); new mints, new checkins, new checkouts blocked until recovery.
- **Data loss target (RPO):** **24 hours** on managed Postgres backup surface.

Free-tier local runtime is not subject to gateway availability because it doesn't depend on the gateway.

3. Measurement + reporting

- **Availability** = $100\% \times (\text{total minutes in period} - \text{unplanned downtime minutes}) / \text{total minutes}$.
- **Planned maintenance** (with ≥ 48 h advance notice) is excluded from downtime.
- **Third-party outages upstream of us** (Render, DO, Cloudflare, Stripe) count toward our downtime — we own the customer relationship, we own the impact.

- **Measurement source:** Render's own service metrics for the gateway; external synthetic monitoring for the mirror (TO BE ADDED — currently monitored ad-hoc via `curl`).

Monthly summary posted internally in incident notes; customer-visible status page is a future item.

4. Failure semantics — how errors are communicated

Consistent HTTP codes matter more than a specific uptime number, because a customer that gets an honest 503 knows to retry, while a customer that gets a masked 200-with-bad-data cannot.

- **4xx:** the customer's request is wrong. 400 malformed, 401 unknown key, 402 payment/capacity, 403 not entitled, 404 not found.
- **503:** service correctly detected an unrecoverable-right-now state (e.g., issuer seed unset, licensing authority unavailable). Retry-safe.
- **5xx (other):** genuine internal error — logged, alerted, treated as an incident per Incident Response Policy §3.
- **Never:** 200 with a fake success payload, empty result silently in place of an error, or silently degraded functionality.

5. Degraded service

Rather than full outages, we optimize for graceful degradation:

- **Free-tier local engine keeps working** when the gateway is down. This is architectural — the engine has no runtime dependency on the gateway.
- **Existing paid entitlements keep working** when the gateway is down. Entitlements are locally-verified against a pinned public key; they only need the gateway for MINTING new ones.
- **Seat checkin fails-open** briefly: an install that can't reach `/v1/seat/checkin` continues with its most recent successful checkin's TTL (30 days per Data Retention §3), not immediately kicked.
- **Webhook retries** are Stripe's job — a temporarily-down `/v1/webhooks/stripe` gets retried by Stripe for 72 hours per their defaults.

6. Planned maintenance

- **Announce ≥ 48 hours in advance** via email to paying customers + Discord post + banner on railcall.ai.
- **Prefer maintenance windows** during low-traffic periods (weekend nights UTC).
- **Zero-downtime deploys preferred:** Render's blue-green pattern for gateway; VPS website `pm2 restart` is ~2 seconds and usually mid-day is fine.
- **Rolling back a bad deploy** doesn't count as planned maintenance (it's an incident recovery per Change Management §7).

7. Communication during incidents

Cross-references Incident Response Policy §8:

- **Any incident affecting the gateway:** post to Discord within 30 min of confirmation, update every hour.
- **Any incident affecting a specific customer:** direct email from `security@railcall.ai`.
- **Post-mortem for any SEV-1 or SEV-2:** shared with affected customers within 5 business days per Incident Response §9.

8. Compensation for SLA breach

Not offered today by default. If a paying customer has an incident-affected month:

- **Sami reviews on request** and typically extends the subscription by the affected days as goodwill. Not automated.

- **Enterprise contracts** may include a formal SLA credit schedule negotiated per-customer.

Explicit non-goal: no financial guarantees baked into the standard `railcall.ai/legal/terms` — the paid tier is a monthly subscription at \$20/seat, sub-guarantee levels appropriate for that price point.

9. Third-party dependency availability

Our availability is bounded by our upstream. See Vendor Management Policy §3 for per-vendor posture. Specifically:

- **Render's SLA:** their published number bounds our gateway availability.
- **DigitalOcean's SLA:** their published number bounds our VPS availability.
- **Cloudflare's SLA:** their published number bounds `railcall.ai` DNS resolution + edge.
- **Stripe's SLA:** their published number bounds new-signup + subscription-lifecycle event availability.

Publishing our own SLA higher than the min of these would be dishonest — the number in §2 respects that.

10. Capacity planning

At current scale (single-digit paying customers, sub-100 free users), capacity is not a limiting factor on any surface. Reviewed quarterly:

- **Gateway response times:** if p99 exceeds 500 ms sustained → investigate (either code, DB indexes, or Render tier upgrade).
- **Postgres row counts:** `consumers`, `seat_reservations`, `processed_events` — projection reviewed quarterly against Render's plan capacity.
- **VPS resources:** `htop` + `df` review during quarterly access review; upgrade droplet size proactively rather than on-demand.

Scaling triggers documented in `~/reviews/YYYY-QN-capacity-review.md`.

11. Review

At each station release + at least annually. Availability numbers in §2 revisited annually against actual measurement — if we consistently beat them, raise the commitment (customers get the honest number); if we consistently miss, lower and investigate why.

12. Related documents

- **BC/DR Policy §3** — RTO/RPO targets this policy commits to publicly.
- **Incident Response Policy §3, §8** — the severity classification + notification mechanics this policy points customers at.
- **Vendor Management Policy §3** — the upstream SLAs bounding our commitment.
- **Change Management Policy §7** — the rollback path that supports the §6 planned-maintenance approach.
- **railcall.ai/legal/terms** — the customer-facing legal version of §2 + §8.